LECTURE 1

AI & ML Foundations, Paradigms and Data Pitfalls

Reference Text

From statistical reasoning to deep learning — and why data must be handled with care

AK DHAMIJA, Researcher & Instructor, AI & Machine Learning

dhamija.ak@gmail.com • 9811453163

Unit 4: Simulation Strategies in Warfare — AI & Machine Learning in Warfare

This reference text expands every slide of the lecture into full explanations, derivations, worked examples and figures. Read it alongside the slides; work the "check your understanding" boxes before looking at the answers.

Contents

1 How to Use This Text **6**

Part A — Data Can Lie **6**

2 Why Data Comes Before Algorithms **7**

3 Simpson's Paradox: Never Add or Average Percentages **8**
3.1 The phenomenon . . . 8
3.2 The kidney-stone study . . . 8
3.3 Why it happens: the confounder . . . 9
3.4 The Berkeley admissions case . . . 9
3.5 The wargaming twin . . . 9

4 Hypothesis Testing and Statistical Significance **10**
4.1 The logic of a significance test . . . 10
4.2 Two kinds of error . . . 10
4.3 Worked example: the radar-absorbent coating trial . . . 11
4.4 Power and sample size . . . 11
4.5 Confidence intervals: a better summary . . . 11

5 Statistical versus Practical Significance, and p-hacking **12**
5.1 Significant is not important . . . 12
5.2 Multiple testing . . . 12

6 Linear Correlation and Its Limits **13**
6.1 Covariance and Pearson's correlation . . . 13
6.2 Anscombe's quartet . . . 13
6.3 Zero correlation does not mean no relationship . . . 13

7 Correlation Is Not Causation **14**
7.1 Controlling for a confounder: partial correlation . . . 14
7.2 Establishing causation . . . 15

8 Regression to the Mean **15**
8.1 The flight-school story . . . 15

9 The Base-Rate Fallacy **16**
9.1 Bayes' theorem . . . 16

10 Missing Data, Selection and Survivorship **17**
10.1 Wald's bombers . . . 17

10.2 Challenger and the excluded flights . . . 17
10.3 Classifying missing data . . . 17

11 AI Interpretation Errors 18
11.1 Shortcut learning: right answer, wrong reason . . . 18
11.2 When humans misread the machine . . . 18
11.3 Calibration . . . 18

12 The Analyst's Oath: Ten Rules 19

Part B — Artificial Intelligence and Machine Learning 19

13 What Is Intelligence? What Is AI? 19
13.1 The four cognitive pillars . . . 19
13.2 The intelligence spectrum . . . 20
13.3 The Turing test . . . 20

14 A Brief History of AI 20

15 Classical AI: Expert Systems and Search 20
15.1 Expert systems . . . 20
15.2 Search: minimax, alpha–beta and A* . . . 21
15.3 The state-space explosion . . . 21
15.4 Why classical AI fails in real warfare . . . 21

16 Intelligent Agents and Their Environments 21
16.1 PEAS: specifying the task . . . 22
16.2 Properties of environments . . . 22
16.3 A hierarchy of agent designs . . . 22

17 Reasoning Under Uncertainty 23
17.1 The rules you need . . . 23
17.2 Combining two sensors: a small Bayesian network . . . 24
17.3 Decisions: expected utility . . . 24

18 AI, Machine Learning and Deep Learning 25
18.1 What it means for a machine to learn . . . 25
18.2 The vocabulary of data . . . 26
18.3 Hypotheses, inductive bias and generalisation . . . 26
18.4 Parametric and non-parametric; discriminative and generative . . . 26
18.5 The machine-learning workflow . . . 27

19 The Five Machine-Learning Paradigms 27

20 Supervised Learning **28**
20.1 Formulation . . . 28
20.2 Loss functions and empirical risk . . . 28
20.3 Gradient descent . . . 28
20.4 Three classic learners, worked by hand . . . 29
20.4.1 Logistic regression: a probability from a weighted sum . . . 29
20.4.2 k-nearest neighbours: judged by the company you keep . . . 29
20.4.3 Decision trees: twenty questions with data . . . 30
20.5 Underfitting, overfitting and model capacity . . . 30
20.6 Evaluation protocol . . . 30
20.7 Metrics from the confusion matrix . . . 31
20.8 Case study: radar threat classification . . . 32
20.9 A map of supervised algorithms . . . 32

21 Unsupervised Learning **33**
21.1 Distance: the hidden decision . . . 33
21.2 k-Means in detail . . . 33
21.3 How many clusters? Elbow and silhouette . . . 34
21.4 Soft clustering: Gaussian mixtures . . . 35
21.5 Dimensionality reduction: PCA . . . 35
21.6 Case study: OPFOR tactic clustering . . . 35

22 Semi-supervised Learning **35**
22.1 Self-training (pseudo-labelling) . . . 36
22.2 Label propagation . . . 36
22.3 Modern consistency methods . . . 36

23 Self-supervised Learning **36**
23.1 Pretext tasks . . . 36
23.2 The contrastive loss . . . 37
23.3 Using the representation . . . 37

24 Reinforcement Learning **37**
24.1 The agent–environment loop . . . 38
24.2 Markov decision processes . . . 38
24.3 Return and discounting . . . 38
24.4 Policies and value functions . . . 39
24.5 The Q-table . . . 39
24.6 The Bellman equations . . . 39
24.6.1 Solving with a model: value iteration . . . 40
24.6.2 Learning without a model . . . 40
24.7 Q-learning . . . 40

24.8 Exploration versus exploitation . . . 41
24.9 On-policy and off-policy: SARSA versus Q-learning . . . 41
24.10 When does Q-learning work? . . . 42
24.11 From tables to networks: the deep Q-network . . . 42
24.11.1 Why tables break . . . 42
24.11.2 Function approximation . . . 42
24.11.3 The two tricks that made DQN work . . . 42
24.11.4 Over-estimation and Double DQN . . . 43
24.12 Beyond value-based RL . . . 43
24.13 Case study: a wargame agent . . . 43

25 Deep Learning **44**
25.1 From the neuron to the network . . . 44
25.2 Training: forward pass, loss, backward pass . . . 45
25.3 Making training work . . . 45
25.4 Convolutional neural networks . . . 46
25.5 Recurrent networks and LSTMs . . . 46
25.6 Attention and Transformers . . . 46
25.7 Autoencoders and generative networks . . . 47
25.8 Why depth helps — and what it costs . . . 47
25.9 Adversarial examples . . . 47

26 Generative AI: LLMs and RAG **48**
26.1 Tokens, embeddings and next-token prediction . . . 48
26.2 Sampling and temperature . . . 48
26.3 From a text predictor to an assistant . . . 48
26.4 Why LLMs hallucinate . . . 49
26.5 Prompting . . . 49
26.6 Retrieval-augmented generation in detail . . . 49
26.7 Security of generative systems . . . 50
26.8 Defence uses and risks . . . 50

27 Defence Applications and Wargaming **50**

28 Data Pitfalls in AI Pipelines **51**
28.1 Survivorship bias and selection . . . 51
28.2 Overfitting and the bias–variance trade-off . . . 51
28.3 Distribution shift . . . 51
28.4 Training-data bias and label noise . . . 52

29 Summary and Glossary **52**
29.1 Summary . . . 52

29.2 Glossary 52

1 How to Use This Text

This reference text accompanies the Lecture 1 slides. Every concept on the slides is explained here in full: what it means, where it comes from, how to compute it, how it is used in defence and wargaming, and how it goes wrong. Coloured boxes mark the different kinds of material:

- **Definition** boxes give precise meanings of terms you must be able to state in an examination.
- **Derivation** boxes show where formulas come from, step by step.
- **Worked example** boxes carry complete numerical calculations.
- **Defence & wargaming** boxes connect the idea to operational practice.
- **Pitfall** boxes describe the most common errors of interpretation.
- **Check your understanding** boxes pose short questions; answers follow immediately so that you can test yourself.

The lecture is organised in two halves. **Part A** deals with the analysis and interpretation of data: the statistical traps that cause intelligent people to draw wrong conclusions. **Part B** introduces artificial intelligence, machine learning and their defence applications. Part A comes first deliberately: an algorithm is only as trustworthy as the data it learns from and the person who interprets its output.

Part A — Data Can Lie

2 Why Data Comes Before Algorithms

Machine-learning algorithms are pattern-finding machines. They do not ask whether the patterns they find are real, causal, representative or relevant to the situation in which they will be used. Those questions are the job of the analyst. Throughout history, the most costly analytical errors have come not from weak mathematics but from data that was collected in a biased way, or interpreted without asking how it came to exist.

The six cases below, drawn from six disciplines, recur throughout this text. Each one has a direct twin in defence analysis and wargaming.

Discipline	What happened	Defence / wargaming twin
Aeronautics (1943)	Engineers proposed armouring bombers where returning aircraft showed bullet holes. Abraham Wald pointed out that the holes marked where an aircraft *could* be hit and survive; the areas without holes were where hits were fatal.	Training an AI only on engagements that were won, or on platforms that returned.
Engineering (1986)	The night before the Challenger launch, O-ring damage was examined against temperature using mainly the flights that *had* damage. Leaving out the damage-free flights hid a strong temperature effect.	Excluding "no-event" sorties from a reliability or vulnerability analysis.
Psychology (1960s)	Israeli Air Force flight instructors believed criticism improved performance and praise worsened it. The pattern was regression to the mean.	Crediting a training aid for the improvement of units that were selected because they had performed worst.
Medicine (1990s)	A pneumonia-risk model learned that asthma *lowered* the risk of death, because asthmatic patients were sent straight to intensive care.	"Sectors with heavy air cover show low losses" — because they received the cover.
Statistics (1973, 1986)	In the Berkeley admissions data and a kidney-stone treatment study, a group that did better in every subgroup did worse in the total: Simpson's paradox.	"AI-equipped units win less often" when they were sent to the hardest sectors.
Defence (1988, 2003)	USS *Vincennes* shot down an airliner its crew believed to be attacking; Patriot batteries engaged two friendly aircraft. Expectations and automated cues were trusted over contrary evidence.	Operators rubber-stamping AI alerts under time pressure.

Key point

Algorithms do not remove these errors — they *automate* them and *scale* them. A biased dataset fed to a deep network produces a confident, fast and biased system.

3 Simpson's Paradox: Never Add or Average Percentages

3.1 The phenomenon

Definition — Simpson's paradox

A trend or comparison that appears in every subgroup of a dataset disappears or reverses when the subgroups are combined. It occurs when a third variable (a *confounder*) is related both to which group an observation falls into and to the outcome, and the groups have different mixes of that variable.

3.2 The kidney-stone study

Charig and colleagues (*British Medical Journal*, 1986) compared open surgery (treatment A) with a less invasive procedure, percutaneous nephrolithotomy (treatment B), for kidney stones. Success counts were:

Stone size	Treatment A	Treatment B
Small	$81/87 = 93\%$	$234/270 = 87\%$
Large	$192/263 = 73\%$	$55/80 = 69\%$
Combined	$273/350 = \mathbf{78}\%$	$289/350 = \mathbf{83}\%$

Treatment A succeeds more often for small stones (93% vs 87%) *and* for large stones (73% vs 69%), yet succeeds less often overall (78% vs 83%).

Derivation — why the combined rate is a weighted average

Let a treatment be given to n_s small-stone and n_ℓ large-stone patients with success rates p_s and p_ℓ. Total successes are $n_s p_s + n_\ell p_\ell$, so the combined rate is

$$p_{\text{all}} = \frac{n_s p_s + n_\ell p_\ell}{n_s + n_\ell} = w\, p_s + (1 - w)\, p_\ell, \qquad w = \frac{n_s}{n_s + n_\ell}.$$

The combined rate is a *weighted* average whose weights are the shares of each subgroup. For A, $w_A = 87/350 = 0.25$: A was mostly given to large stones. For B, $w_B = 270/350 = 0.77$: B was mostly given to small stones. Thus

$$p_A = 0.25(0.93) + 0.75(0.73) = 0.78, \qquad p_B = 0.77(0.87) + 0.23(0.69) = 0.83.$$

A's combined rate is pulled toward the low large-stone rate because three-quarters of its patients had large stones.

Pitfall — adding or averaging percentages

It is tempting to add the subgroup percentages ($93 + 73 = 166$ for A, $87 + 69 = 156$ for B) or to average them (83% vs 78%), which would wrongly suggest that A is better overall by five points. Both operations are meaningless because they give each subgroup equal weight regardless of how many patients it contains. **Totals must be computed from counts.** Every reported percentage should be accompanied by its numerator and denominator.

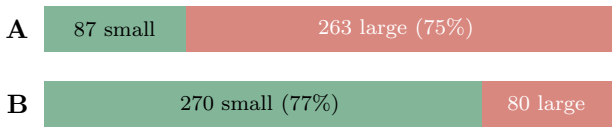


Figure 1: case mix. Large stones are harder to treat and were mostly assigned to A.

3.3 Why it happens: the confounder

Stone size is a *confounder*: it influences both the choice of treatment (surgeons chose open surgery for large stones) and the outcome (large stones are harder to treat). Because the two treatment groups have very different stone-size mixes, the overall comparison mixes the effect of the treatment with the effect of stone size. The fair comparison is *within* each stone size, where A is better.

3.4 The Berkeley admissions case

In 1973 the University of California, Berkeley admitted about 44% of male graduate applicants but only 35% of female applicants. Bickel, Hammel and O'Connell (*Science*, 1975) showed that department by department there was no bias against women. Women had applied disproportionately to the most competitive departments, which rejected most applicants of either sex. The department applied to was the confounder.

3.5 The wargaming twin

Consider exercise records for AI-assisted and conventional units, split by terrain (illustrative teaching data):

Terrain	AI-assisted units	Conventional units
Open / easy	$81/90 = 90\%$	$238/280 = 85\%$
Mountain / hard	$105/210 = 50\%$	$18/40 = 45\%$
All terrain	$186/300 = \mathbf{62}\%$	$256/320 = \mathbf{80}\%$

A headline "AI-assisted units win only 62% vs 80%" would invite withdrawing the AI tool. Yet AI-assisted units win more in each terrain type. They were assigned to the hard mountain sectors: 210 of their 300 engagements (70%) compared with 40 of 320 (12.5%) for conventional units.

Defence & wargaming application — detecting Simpson's paradox in exercise data

Before comparing any two groups — units, platforms, tactics, software versions — ask: *what decided which observations ended up in each group?* If that factor (terrain, threat level, time of day, unit experience) also affects the outcome, stratify by it, or use a design in which assignment is randomised. In AI evaluation, the same logic applies when comparing model accuracy across datasets with different difficulty mixes.

Check your understanding — Simpson's paradox

A sensor upgrade is fitted to 120 aircraft that fly mostly night sorties (90 night, 30 day). Detection success: night 54/90, day 27/30. A comparison fleet of 120 non-upgraded aircraft flies 20 night and 100 day sorties: night 10/20, day 85/100. (a) Compute all subgroup and overall rates. (b) Which fleet does better overall, and which within each condition? (c) What is the confounder?

Answer. (a) Upgraded: night 60%, day 90%, overall 81/120 = 67.5%. Non-upgraded: night 50%, day 85%, overall 95/120 = 79.2%. (b) The upgrade is better at night (+10 points) and by day (+5 points) but worse overall. (c) Sortie type (night vs day): night detection is harder, and the upgraded fleet flies mostly at night.

4 Hypothesis Testing and Statistical Significance

4.1 The logic of a significance test

A hypothesis test asks whether the data are surprising enough to doubt a default assumption.

Definition — null and alternative hypotheses

The **null hypothesis** H_0 is the default claim of "no effect" or "no difference" (e.g. a new coating does not change the detection probability). The **alternative hypothesis** H_1 is the claim we would accept if the data are inconsistent with H_0 (e.g. the coating changes the detection probability).

The steps are always the same: (1) state H_0 and H_1 and the significance level α *before* looking at the data; (2) choose a test statistic whose distribution under H_0 is known; (3) compute it from the data; (4) compute the p-value; (5) reject H_0 if $p < \alpha$, otherwise do not reject it.

Definition — p-value

The p-value is the probability, *assuming H_0 is true*, of obtaining a test statistic at least as extreme as the one observed. It is **not** the probability that H_0 is true, **not** the probability that the result happened "by chance", and **not** a measure of the size or importance of an effect.

4.2 Two kinds of error

	H_0 true (no real effect)	H_0 false (real effect)
Reject H_0	**Type I error** (false alarm), probability α	Correct; probability $1-\beta$ = **power**
Do not reject	Correct	**Type II error** (missed effect), probability β

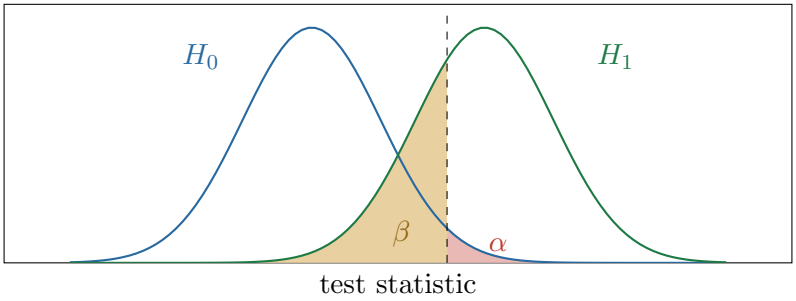


Figure 2: distributions of the test statistic under H_0 and H_1. Moving the critical value trades α against β; more data separates the curves and reduces both.

4.3 Worked example: the radar-absorbent coating trial

Worked example — two-proportion z-test

A trial flies 40 sorties with the baseline coating (detected in 26: $\hat{p}_1 = 0.65$) and 40 with a new coating (detected in 18: $\hat{p}_2 = 0.45$). H_0: $p_1 = p_2$; H_1: $p_1 \neq p_2$; $\alpha = 0.05$.
Pooled proportion under H_0: $\hat{p} = (26 + 18)/80 = 0.55$.
Standard error under H_0: $SE = \sqrt{\hat{p}(1-\hat{p})\left(\frac{1}{n_1} + \frac{1}{n_2}\right)} = \sqrt{0.55 \times 0.45 \times 0.05} = \sqrt{0.012375} = 0.111$.
Test statistic: $z = (0.65 - 0.45)/0.111 = 1.80$.
p-value (two-sided): $p = 2\,[1 - \Phi(1.80)] = 2(0.0359) = 0.072$.
Since $p > 0.05$ we do **not** reject H_0.

Pitfall — "not significant" does not mean "no effect"

Concluding that "the coating does not work" is wrong. A 20-point reduction in detection is operationally large; the trial was simply too small to distinguish it reliably from chance. **Absence of evidence is not evidence of absence.** The right conclusion is "inconclusive: more sorties needed".

4.4 Power and sample size

Derivation — sample size for comparing two proportions

To detect a difference $\Delta = p_1 - p_2$ with significance α (two-sided) and power $1 - \beta$, the approximate number per group is

$$n \approx \frac{\left(z_{1-\alpha/2} + z_{1-\beta}\right)^2 \left[p_1(1-p_1) + p_2(1-p_2)\right]}{\Delta^2}.$$

For $\alpha = 0.05$ ($z = 1.96$) and 80% power ($z = 0.84$), $p_1 = 0.65$, $p_2 = 0.45$:

$$n \approx \frac{(2.80)^2(0.2275 + 0.2475)}{0.20^2} = \frac{7.84 \times 0.475}{0.04} \approx 93 \text{ sorties per arm.}$$

The trial ran 40 per arm — well under half of what was needed.

Key point

Always compute the required sample size *before* running a trial or an exercise. An underpowered trial wastes resources, because a "non-significant" result cannot distinguish "no effect" from "too little data".

4.5 Confidence intervals: a better summary

A confidence interval reports the range of effect sizes compatible with the data. For a difference in proportions, using the unpooled standard error,

$$SE_u = \sqrt{\frac{0.65(0.35)}{40} + \frac{0.45(0.55)}{40}} = \sqrt{0.011875} = 0.109,$$

$$95\%\ \text{CI} = 0.20 \pm 1.96(0.109) = (-0.014,\ 0.414).$$

The interval runs from essentially no effect to a 41-point reduction. It tells the decision-maker far more than "$p = 0.072$": the data are compatible with a negligible effect *and* with a very large one.

5 Statistical versus Practical Significance, and p-hacking

5.1 Significant is not important

With enough data, trivially small effects become "statistically significant". Suppose one million simulated engagements are run for each of two tactics, with hit rates 50.0% and 50.2%. The standard error of the difference is $\sqrt{0.25 \times 2/10^6} = 0.0007$, so $z = 0.002/0.0007 \approx 2.8$ and $p \approx 0.005$. The result is "highly significant", yet a 0.2-point difference would never change a tactical decision.

Key point

Always report the **effect size** (difference, ratio, or standardised effect such as Cohen's d) with a confidence interval, and ask whether it matters operationally. Statistical significance answers "is it probably not zero?"; practical significance answers "is it big enough to matter?".

5.2 Multiple testing

If m independent tests are run when no real effects exist, the probability that at least one gives $p < \alpha$ is

$$P(\geq 1 \text{ false positive}) = 1 - (1 - \alpha)^m.$$

For $\alpha = 0.05$ and $m = 20$ this is $1 - 0.95^{20} = 0.64$: a 64% chance of a spurious "discovery". Testing many tactics, sectors, time windows or features and reporting only the significant one is called **p-hacking** or data dredging.

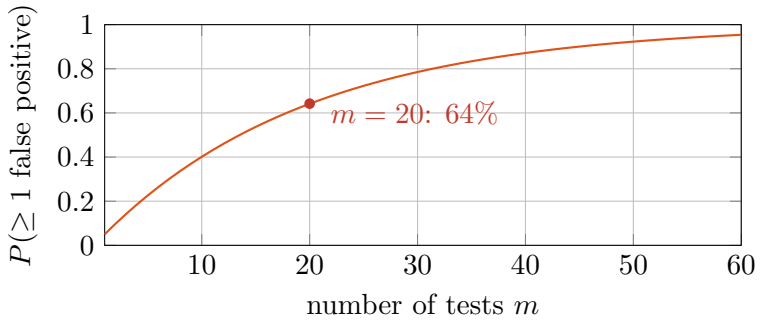


Figure 3: the family-wise error rate grows quickly with the number of tests.

Remedies. (1) *Bonferroni correction*: use α/m per test, which keeps the family-wise rate at or below α (conservative). (2) *Pre-registration*: state the hypothesis and analysis before seeing the data. (3) *Replication*: confirm any discovery on fresh data.

Defence & wargaming application — the replication crisis as a warning

When 100 published psychology experiments were repeated, only about 36% gave a significant result again (Open Science Collaboration, *Science*, 2015). Small samples, flexible analysis and the publication of only "significant" findings produced a literature full of false positives. Wargame analysis and AI evaluation are exposed to the same pressures: many configurations are tried and the best is reported. Treat any single-run result as a hypothesis to be confirmed.

6 Linear Correlation and Its Limits

6.1 Covariance and Pearson's correlation

Definition — Pearson correlation coefficient

For paired observations (x_i, y_i), $i = 1, \ldots, n$,

$$r = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2}\,\sqrt{\sum_i (y_i - \bar{y})^2}} \in [-1, 1].$$

$r = +1$: perfect increasing straight line; $r = -1$: perfect decreasing straight line; $r = 0$: no *linear* association. r^2 is the proportion of the variance of y explained by the best straight line.

The numerator is n times the sample covariance: it is positive when above-average x values tend to go with above-average y values. Dividing by the two standard deviations makes r unit-free.

6.2 Anscombe's quartet

In 1973 the statistician Francis Anscombe built four datasets of eleven points each that share almost identical summary statistics: mean of $x = 9$, mean of $y = 7.50$, equal variances, $r = 0.816$, and the same least-squares line $y = 3.00 + 0.500x$. Plotted, they look completely different (Figure 4): one is a genuine noisy line, one is a smooth curve, one is a perfect line spoiled by a single outlier, and one has all points at the same x except one leverage point that creates the entire "correlation".

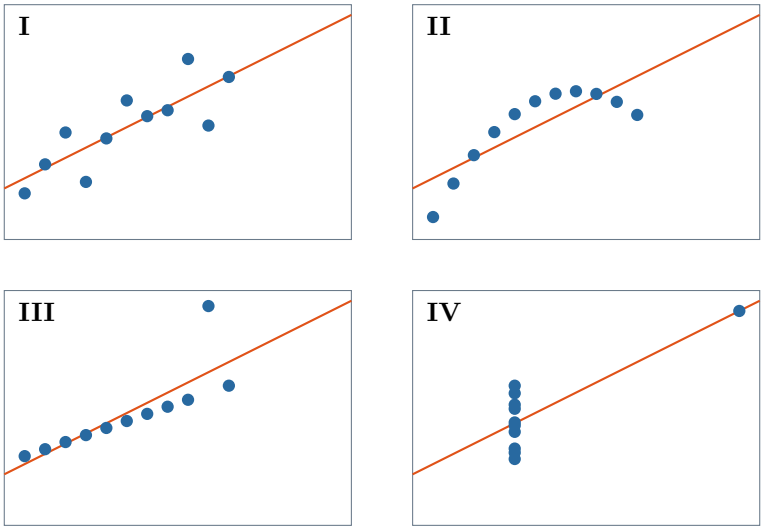


Figure 4: Anscombe's quartet. All four have $r = 0.816$ and the same fitted line.

6.3 Zero correlation does not mean no relationship

Correlation measures only straight-line association. Total aerodynamic drag on an aircraft falls and then rises with airspeed: induced drag decreases with speed while parasite drag increases. Across the flight envelope the U-shaped relationship is perfectly deterministic, yet its linear correlation can be close to zero. A rank correlation (Spearman's ρ, the Pearson correlation of the ranks) captures monotonic but non-linear relationships; neither captures U-shapes.

Pitfall — trusting a single summary number

A correlation, a regression slope, or an AI feature-importance score is a summary that can be produced by very different data. **Always plot the data first.** One outlier or one leverage point can manufacture or destroy a correlation.

7 Correlation Is Not Causation

A correlation between X and Y can arise in four ways other than "X causes Y":

1. **Confounding (common cause).** A third variable Z drives both. Across 17 European countries the number of breeding stork pairs correlates with the birth rate ($r \approx 0.62$, Matthews 2000); both are related to land area and rural population. Ice-cream sales and drownings both rise in summer.
2. **Reverse causation.** Y causes X. Vehicles with the most maintenance hours have the most breakdowns — not because maintenance causes failure, but because failing vehicles attract maintenance.
3. **Selection (collider) bias.** The data were selected on a variable affected by both X and Y. If only engagements that were "reported upward" are analysed, and reporting depends both on unit type and on outcome, a spurious association between unit type and outcome is created.
4. **Chance.** With enough variables, some will correlate by accident. Screening 1,000 indicators with no real effect at $\alpha = 0.05$ produces about 50 "significant" correlations.

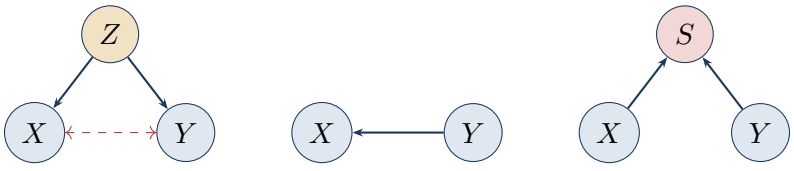


Figure 5: three causal structures that produce correlation without X causing Y.

7.1 Controlling for a confounder: partial correlation

Definition — partial correlation

The correlation between X and Y after removing the linear effect of Z from both:

$$r_{XY\cdot Z} = \frac{r_{XY} - r_{XZ}r_{YZ}}{\sqrt{(1 - r_{XZ}^2)(1 - r_{YZ}^2)}}.$$

Worked example — the catering paradox

Over three years of wargame data, a "catering-quality score" correlated with exercise win rate at $r = 0.71$. Controlling for unit budget/readiness, the partial correlation fell to $r_{\text{partial}} = 0.09$ ($p = 0.42$). Well-funded units both ate better and trained better; catering did not win battles.

7.2 Establishing causation

The gold standard is a **randomised controlled experiment**: assignment to treatment is random, so confounders are balanced on average. In wargaming this means randomly assigning tactics, tools or units to scenarios (an A/B wargame). When experiments are impossible, causal claims need a design that blocks confounders (stratification, matching, regression adjustment, natural experiments) together with a plausible mechanism, consistency across settings, a dose–response relationship, and the right temporal order — considerations formalised in epidemiology by Bradford Hill (1965).

8 Regression to the Mean

8.1 The flight-school story

Daniel Kahneman recounts that instructors at an Israeli Air Force flight school observed that cadets praised after an excellent landing usually did worse on the next one, while cadets criticised after a poor landing usually improved. The instructors concluded that criticism works and praise does not. The real explanation is statistical.

Derivation — why extreme scores move toward the mean

Model each observed score as true skill plus luck: $X_1 = T + e_1$, $X_2 = T + e_2$, with T, e_1, e_2 independent, $\mathbb{E}[e] = 0$, $\mathrm{Var}(T) = \sigma_T^2$, $\mathrm{Var}(e) = \sigma_e^2$. The best linear prediction of the second score from the first is

$$\mathbb{E}[X_2 \mid X_1] = \mu + \rho\,(X_1 - \mu), \qquad \rho = \frac{\sigma_T^2}{\sigma_T^2 + \sigma_e^2} < 1.$$

Because $\rho < 1$ whenever luck plays any part, a cadet who scored far above the mean is expected to score closer to the mean next time — with no intervention at all. The more luck in the measurement, the stronger the regression.

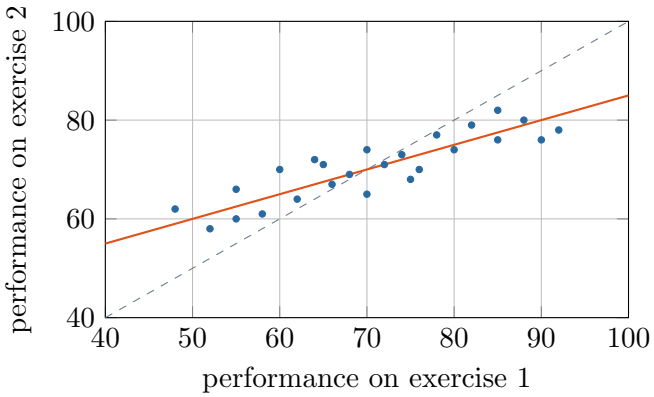


Figure 6: regression to the mean (illustrative). The fitted slope is less than the 45° line.

The word "regression" itself comes from Francis Galton (1886), who found that children of very tall parents are tall but less extreme — "regression towards mediocrity".

Defence & wargaming application — evaluating training interventions

"The five worst-scoring battalions improved after receiving the new AI training aid." They would be expected to improve anyway. A valid evaluation needs a **control group**: equally poor units, chosen the same way, that do not receive the aid. The effect of the aid is the

difference in improvement between the two groups.

9 The Base-Rate Fallacy

9.1 Bayes' theorem

Derivation — positive predictive value

Let T = "track is a real threat" with prior probability (base rate) $p = P(T)$. A detector has sensitivity $s = P(\text{alarm} \mid T)$ and specificity $c = P(\text{no alarm} \mid \bar{T})$. By Bayes' theorem,

$$\text{PPV} = P(T \mid \text{alarm}) = \frac{P(\text{alarm} \mid T)P(T)}{P(\text{alarm})} = \frac{s\,p}{s\,p + (1-c)(1-p)}.$$

Worked example — a "99% accurate" threat detector

Out of 10,000 air tracks, 10 are real threats ($p = 0.001$). The detector has $s = c = 0.99$.

- Real threats detected: $0.99 \times 10 \approx 10$ alarms (0.1 threat missed).
- Non-threats falsely flagged: $0.01 \times 9{,}990 \approx 100$ alarms.
- $\text{PPV} = \dfrac{0.99 \times 0.001}{0.99 \times 0.001 + 0.01 \times 0.999} = \dfrac{0.00099}{0.01098} \approx \mathbf{9}\%$.

Ten of every eleven alarms are false, even though the detector is "99% accurate".

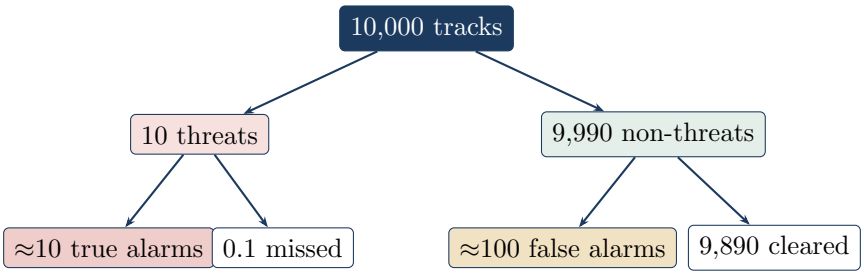


Figure 7: the natural-frequency tree makes the base-rate effect obvious.

The same arithmetic applies in medicine: a screening test with 99% sensitivity and specificity for a disease with 1% prevalence gives $\text{PPV} = 0.0099/(0.0099 + 0.0099) = 50\%$. Studies repeatedly find that clinicians over-estimate this figure.

Pitfall — judging alarms without the base rate

A brochure accuracy figure measured on balanced test data says nothing about how often an alarm is real in the deployment stream. Always ask for precision (PPV) *at the operational base rate*. Low PPV produces alarm fatigue (operators ignore alerts) or over-reaction (everything is engaged). Mitigation: a second-stage check by another sensor or a human for every alarm.

10 Missing Data, Selection and Survivorship

10.1 Wald's bombers

In 1943 the Statistical Research Group at Columbia studied the distribution of damage on returning bombers. Engineers wanted to add armour where holes were densest — wings and fuselage. Abraham Wald argued that the sample contained only aircraft that returned. Aircraft hit in the engines or cockpit were missing from the data because they had been lost. The absence of holes in those areas was the signal: armour belongs where returning aircraft show *no* damage. This is **survivorship bias**: the process that produced the data filtered out the most important cases.

10.2 Challenger and the excluded flights

Before the 1986 Challenger launch, O-ring damage was discussed against launch temperature using mainly the flights that showed damage. Those points showed no obvious trend. Including the damage-free flights reveals the pattern: every flight below about 66°F had O-ring damage, while most warm flights had none. Challenger launched at about 31°F — colder than any previous flight — and was lost with its crew of seven. Two errors compounded: excluding the "no-event" cases removed the contrast that reveals a trend, and the decision extrapolated far beyond the observed temperature range.

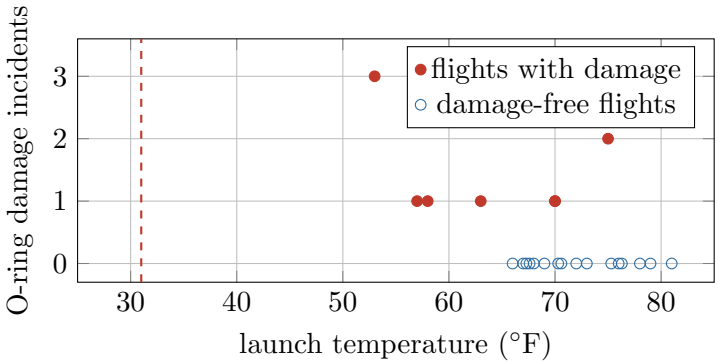


Figure 8: approximate reconstruction of the pre-launch flight record (cf. Dalal, Fowlkes & Hoadley, *JASA*, 1989). Dashed line: launch-day forecast.

10.3 Classifying missing data

Definition — missing-data mechanisms

MCAR (missing completely at random): missingness is unrelated to anything — e.g. a random sensor dropout. **MAR** (missing at random): missingness depends on observed variables — e.g. IR data missing more often in recorded cloudy weather. **MNAR** (missing not at random): missingness depends on the missing value itself — e.g. destroyed platforms never report their damage. MNAR is the dangerous case: no amount of data from the survivors corrects it.

11 AI Interpretation Errors

11.1 Shortcut learning: right answer, wrong reason

- **Clever Hans (1907).** A horse appeared to perform arithmetic; the psychologist Oskar Pfungst showed it responded to tiny involuntary cues from its questioner. Machine-learning models do the same: they exploit whatever correlates with the label.
- **Husky vs wolf.** A classifier separated wolves from huskies using snow in the background (Ribeiro et al., 2016). Explanation tools (LIME) exposed it.
- **Pneumonia risk.** A model rated asthmatic patients as lower risk because they were treated more aggressively (Caruana et al., 2015) — confounding by treatment.
- **Adversarial examples.** A few stickers on a stop sign made a vision model read it as a speed-limit sign (Eykholt et al., 2018). An adversary can design camouflage against a known classifier.
- **Metadata leakage.** A model "predicts" night attacks from timestamps rather than thermal signatures; it fails when the adversary changes schedule.
- The often-told story of a network that "detected tanks" by recognising cloudy skies is probably apocryphal, but the failure mode it describes is real.

11.2 When humans misread the machine

Definition — automation bias

The tendency to accept an automated recommendation without sufficient scrutiny, especially under time pressure or high workload — including failing to act on information that contradicts the automation.

In 2003 Patriot batteries shot down a British Tornado and a US Navy F/A-18, killing three aircrew; contributing factors included misclassification of friendly aircraft and procedures in which crews accepted automated engagement cues. In 1988 the crew of USS *Vincennes*, under combat stress, perceived Iran Air 655 as a descending attacking aircraft although the data showed a climbing airliner; 290 people died. Psychologists call this *expectation bias* or scenario fulfilment.

11.3 Calibration

Definition — calibration

A classifier is **calibrated** if, among all cases where it reports confidence q, the fraction that are correct is q. The *expected calibration error* groups predictions into confidence bins B_m:

$$\mathrm{ECE} = \sum_{m} \frac{|B_m|}{n} \left|\mathrm{acc}(B_m) - \mathrm{conf}(B_m)\right|.$$

Modern deep networks are often over-confident: a "99% confident" output may be correct only about 80% of the time, and worse under distribution shift. Display calibrated uncertainty rather than bare labels, and never treat confidence as correctness.

12 The Analyst's Oath: Ten Rules

1. **Count, don't average percentages.** Totals come from raw numbers.
2. **Ask what is missing.** Survivors, damage-free flights, unreported engagements.
3. **Stratify by what drove assignment.** Terrain, severity, budget, threat level.
4. **State H_0 and compute power before testing.**
5. **Report effect sizes with confidence intervals**, not only p-values.
6. **Correct for multiple tests** and confirm on fresh data.
7. **Plot the data.** One number hides the shape.
8. **Treat correlation as a question, not an answer.** Seek a mechanism and an experiment.
9. **Check the base rate** before trusting any alarm or accuracy figure.
10. **Test on the deployment domain and keep a human accountable.**

Part B — Artificial Intelligence and Machine Learning

13 What Is Intelligence? What Is AI?

13.1 The four cognitive pillars

Intelligent behaviour can be decomposed into four capabilities that form a closed loop:

- **Perceive** — sense and interpret the environment (vision, hearing, radar, language), integrating several modalities.
- **Reason** — draw logical inferences, analyse cause and effect, form hypotheses.
- **Learn** — adapt from experience and generalise to new situations by reducing prediction errors.
- **Act** — choose and execute a response, then evaluate the outcome, which feeds back into perception.

A Blue-Force AI agent in a wargame must do all four: perceive the battle map through sensor fusion, reason about OPFOR intent, learn from earlier engagements, and act to maximise mission success.

Definition — artificial intelligence

McCarthy (1956): "the science and engineering of making intelligent machines, especially intelligent computer programs." **Russell & Norvig (2020):** AI studies *agents* that receive percepts from an environment and perform actions; each agent implements a function f : percept sequences $\rightarrow$ actions.

13.2 The intelligence spectrum

Human intelligence ⊃ artificial general intelligence (AGI, hypothetical human-level AI across all domains) ⊃ narrow AI (excellent at one task, no transfer) ⊃ machine learning ⊃ deep learning. Every deployed AI system today is **narrow AI**: superb at a specific task, without common sense or cross-domain reasoning. Treating a narrow system as if it understood the situation is a root cause of many failures.

13.3 The Turing test

Alan Turing (1950) proposed the "imitation game": an interrogator exchanges typed messages with a hidden human and a hidden machine and must decide which is which. A machine that cannot be reliably distinguished "passes". Modern large language models often pass text-only versions, but passing shows convincing imitation, not understanding. For defence, what matters is *task performance* under realistic conditions. The defence angle runs the other way too: an adversary's AI that blends into legitimate communications, or deep-fake voices used for deception, are threats to be detected.

14 A Brief History of AI

Year	Milestone
1950	Turing, "Computing Machinery and Intelligence": the imitation game.
1956	Dartmouth summer project; McCarthy coins "artificial intelligence".
1965–80	Expert-system era: MYCIN (medical diagnosis), XCON (computer configuration); rule-based AI peaks.
1986	Rumelhart, Hinton & Williams popularise back-propagation: neural networks revive.
1997	IBM Deep Blue defeats Kasparov using search plus expert heuristics.
2006	Hinton's deep belief networks; the deep-learning renaissance begins with GPU computing.
2012	AlexNet wins the ImageNet challenge: convolutional networks transform image analysis.
2017	"Attention Is All You Need" introduces the Transformer.
2022	ChatGPT and GPT-4; generative AI enters mainstream and defence-intelligence use.

Each wave followed the removal of a bottleneck: hand-written rules (1960s), learning algorithms and data (1980s), GPU compute and large labelled datasets (2012), attention and scale (2017 onwards). Periods of over-promising were followed by "AI winters" of reduced funding.

15 Classical AI: Expert Systems and Search

15.1 Expert systems

An expert system encodes human expertise as IF–THEN rules. Its architecture has a **knowledge base** (rules elicited from experts), **working memory** (facts about the current situation, e.g. from sensors), an **inference engine** that applies rules, and an **explanation facility** that reports which rules fired.

- **Forward chaining** starts from facts and fires rules until conclusions are reached: data-driven.
- **Backward chaining** starts from a goal and searches for facts that support it: goal-driven.

Worked example — IADS threat rule

Rule R1: `IF RadarSig = LARGE AND Speed > 800 km/h AND Altitude < 300 m THEN Threat = HIGH`. Facts: Speed = 900 km/h, Altitude = 200 m, Sig = LARGE. Forward chaining fires R1 and concludes Threat = HIGH; the explanation facility reports the three conditions — full auditability.

Strengths: transparent, auditable, no training data, deterministic. **Weaknesses:** the knowledge-acquisition bottleneck (every rule hand-written), and brittleness. In the lecture's artillery fire-assignment example, an expert system with 500 rules (200 expert-months) was correct in 91.2% of known-doctrine cases but only 19.0% of novel scenarios, where a random forest trained on data achieved 74.3%. War is inherently novel.

15.2 Search: minimax, alpha–beta and A*

In two-player zero-sum games MAX (Blue) maximises and MIN (Red) minimises the evaluation of the final position. **Minimax** evaluates the game tree to depth d with branching factor b: cost $O(b^d)$. **Alpha–beta pruning** skips branches that cannot change the decision; with good move ordering the cost falls to $O(b^{d/2})$, doubling the reachable depth.

Worked example — an alpha–beta cut-off

MAX has three moves. The left MIN node sees leaves 3 and 5, so its value is 3. In the middle MIN node the first leaf is 2. Since MIN can force at most 2, which is already worse for MAX than the 3 available on the left, the remaining middle leaf need not be examined: it is pruned.

A* search finds least-cost paths by expanding nodes in order of $f(n) = g(n) + h(n)$, where g is the cost so far (terrain, fuel, time) and h is a heuristic estimate of the remaining cost. If h never over-estimates (admissible), A* returns an optimal path.

15.3 The state-space explosion

A wargame with 8 units on a 50×50 hex grid, each with 6 actions, has $b_{\text{eff}} = 6^8 \approx 1.68$ million joint actions per turn. Joint search to depth 4 needs about $(6^8)^4 \approx 8 \times 10^{24}$ nodes — infeasible. Factoring the search unit by unit (6 actions, depth 4) needs only $6^4 = 1{,}296$ nodes, at the price of ignoring coordination. Realistic wargames have on the order of 10^{80} states.

15.4 Why classical AI fails in real warfare

(1) Brittleness: novel situations break the rules. (2) The knowledge bottleneck: encoding doctrine by hand is slow and incomplete. (3) No uncertainty handling: weather, morale and communications loss are probabilistic. (4) State-space explosion. (5) No learning from experience. The remedy is to **learn from data** — machine learning.

16 Intelligent Agents and Their Environments

Before studying learning algorithms it helps to be precise about *what* an AI system is. Modern AI is organised around the idea of an **agent**: anything that perceives its environment through sensors and acts on it through actuators. A radar-cueing system, a chess program, a mine-hunting drone and a large language model answering an analyst's query are all agents in this sense. The agent view forces three questions that every defence AI project must answer: what can the system sense, what can it do, and how will we judge whether it did well?

16.1 PEAS: specifying the task

A task environment is specified by four elements, abbreviated **PEAS**: the **P**erformance measure, the **E**nvironment, the **A**ctuators and the **S**ensors.

	Air-defence threat evaluator	Mine-hunting UUV	Wargame OPFOR agent
Performance	threats engaged in time; fratricide = 0; false alarms per hour	area cleared per hour; mines missed; vehicle survival	Blue losses inflicted; objectives held; doctrine fidelity
Environment	airspace, weather, friendly and civil traffic, jamming	sea-bed, currents, turbidity, clutter	terrain, Blue forces, fog of war
Actuators	alert, recommend engagement, cue sensors	thrusters, sonar ping, marker drop	move, fire, hide, call support
Sensors	radar tracks, IFF, ESM, data links	side-scan sonar, inertial navigation, depth	simulated line-of-sight, reports

Writing the PEAS table first is the cheapest way to discover that a project is ill-posed. If the performance measure cannot be written down, no learning algorithm can optimise it; if a decisive variable is not among the sensors, no amount of data will let the agent infer it.

16.2 Properties of environments

Environments differ along six dimensions, and each dimension pushes the choice of method:

- **Fully vs partially observable.** Chess is fully observable; a battlefield is not (fog of war). Partial observability calls for memory, belief states and filtering (Kalman filters in Lecture 3, POMDPs in Lecture 10).
- **Deterministic vs stochastic.** If the next state is completely determined by the current state and action, planning can be exact; weapon effects, weather and human behaviour are stochastic.
- **Episodic vs sequential.** Classifying one radar pulse does not affect the next classification (episodic): supervised learning suffices. Manoeuvring a unit changes what happens later (sequential): reinforcement learning or planning is needed.
- **Static vs dynamic.** Does the world change while the agent deliberates? An incoming missile does; a terrain map does not.
- **Discrete vs continuous.** Hex-grid moves are discrete; drone thrust is continuous (Lecture 11).
- **Single- vs multi-agent; cooperative vs competitive.** A wargame has teammates and an adversary that adapts (Lecture 12).

Real warfare is the hardest corner of this space: partially observable, stochastic, sequential, dynamic, continuous and multi-agent with an adversary who is actively trying to deceive the sensors.

16.3 A hierarchy of agent designs

1. **Simple reflex agents** map the current percept to an action through condition–action rules (`IF track is hostile AND in range THEN alert`). They are fast and transparent but fail whenever the right action depends on history.

2. **Model-based reflex agents** keep an internal state — a model of how the world evolves — so they can act sensibly when the current percept is incomplete (a track that has just dropped out of radar coverage is still there).
3. **Goal-based agents** search or plan for action sequences that reach a goal (A* route planning).
4. **Utility-based agents** choose actions that maximise expected utility, trading off competing goals (speed against risk) under uncertainty.
5. **Learning agents** contain a *performance element* (one of the above), a *critic* that compares outcomes with a standard, a *learning element* that improves the performance element, and a *problem generator* that proposes exploratory actions. Every machine-learning system in this course is the learning element of such an agent.

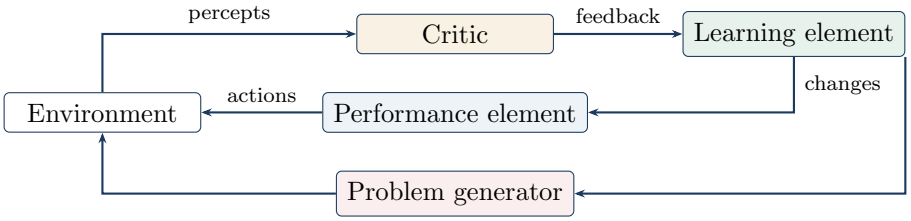


Figure B1: the general learning agent. The critic needs a fixed performance standard — in RL it is the reward.

17 Reasoning Under Uncertainty

Classical rule-based systems assume facts are either true or false. Real sensors deliver evidence of varying reliability, and the right conclusion depends on how evidence combines with prior knowledge. Probability theory is the calculus for this, and it underlies almost every learning method in the course.

17.1 The rules you need

For events A and B:

$$P(A \cap B) = P(A \mid B)\,P(B), \qquad P(A) = \sum_{b} P(A \mid B = b)P(B = b),$$

$$P(A \mid B) = \frac{P(B \mid A)P(A)}{P(B)}.$$

The last line is **Bayes' rule**. In words: *posterior* $\propto$ *likelihood* $\times$ *prior*. The prior encodes what we believed before the evidence (the base rate of threats); the likelihood says how probable the evidence is under each hypothesis (the sensor's characteristics); the posterior is the updated belief. Section 9 applied this to a single detector; here we extend it to several sources of evidence.

17.2 Combining two sensors: a small Bayesian network

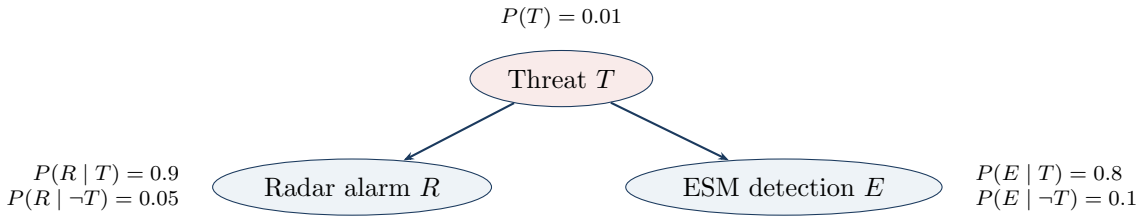


Figure B2: a three-node Bayesian network. Arrows point from cause to evidence.

A **Bayesian network** is a directed acyclic graph whose nodes are random variables and whose arrows encode direct dependence. The graph in Figure B2 asserts that radar and ESM respond to the threat but not to each other once the threat status is known (they are *conditionally independent* given T). The joint distribution then factorises as $P(T, R, E) = P(T)P(R \mid T)P(E \mid T)$, which needs 5 numbers instead of the 7 required for an arbitrary joint distribution over three binary variables. For n sensors the saving is exponential.

Worked example — posterior threat probability after one and two alarms

Radar alone. $P(T \mid R) = \dfrac{0.01 \times 0.9}{0.01 \times 0.9 + 0.99 \times 0.05} = \dfrac{0.009}{0.009 + 0.0495} = 0.154$. One alarm lifts the threat probability from 1% to 15%: still more likely a false alarm.

Radar and ESM. $P(T \mid R, E) = \dfrac{0.01 \times 0.9 \times 0.8}{0.01 \times 0.9 \times 0.8 + 0.99 \times 0.05 \times 0.1} = \dfrac{0.0072}{0.0072 + 0.00495} = 0.593$. Two independent, individually weak pieces of evidence together make the threat more likely than not.

Lesson. Independent evidence multiplies likelihood ratios: the radar's ratio is $0.9/0.05 = 18$, the ESM's $0.8/0.1 = 8$; prior odds $1 : 99$ times $18 \times 8 = 144$ give posterior odds $144 : 99$, i.e. 0.593. If the two sensors were in fact correlated (both fooled by the same decoy), multiplying would overstate the evidence. Assuming independence that does not exist is the same error that makes Naive Bayes over-confident (Lecture 2).

Larger Bayesian networks are used for intelligence fusion (indicators and warnings), equipment fault diagnosis and threat assessment. They are transparent — every number has a meaning that an analyst can challenge — and they can be learned from data, elicited from experts, or both. They bridge classical AI (explicit knowledge) and machine learning (parameters estimated from data).

17.3 Decisions: expected utility

Probabilities alone do not decide anything; they must be combined with the consequences of each action. A rational agent chooses the action that maximises expected utility $EU(a) = \sum_s P(s \mid \text{evidence})\, U(a, s)$.

Worked example — to engage or not

With $P(T \mid R, E) = 0.593$, suppose utilities (on an arbitrary scale) are: engage a real threat $+100$; engage a non-threat (possible civil aircraft) -500; hold fire on a real threat -300; hold fire on a non-threat 0. Then $EU(\text{engage}) = 0.593(100) + 0.407(-500) = -144.2$ and $EU(\text{hold}) = 0.593(-300) + 0.407(0) = -177.9$. Engaging has the higher expected utility here, but only narrowly; with a fratricide cost of -600 the decision flips ($EU(\text{engage}) = -184.9$). The choice depends as much on the *value judgements* in the utility table as on the probabilities — and those value judgements belong to commanders and the law, not to the algorithm.

Key point

Machine learning estimates probabilities; decision-making also needs utilities. Keep the two separate in any defence AI design, so that the value judgements remain visible, auditable and under human control.

18 AI, Machine Learning and Deep Learning

Definition — the nested hierarchy

AI is any technique enabling machines to perform tasks associated with intelligence (rules, search, planning, probabilistic reasoning, learning). **Machine learning** is the subset of AI in which systems improve from data without being explicitly programmed. **Deep learning** is the subset of ML using multi-layer neural networks that learn hierarchical representations directly from raw data.

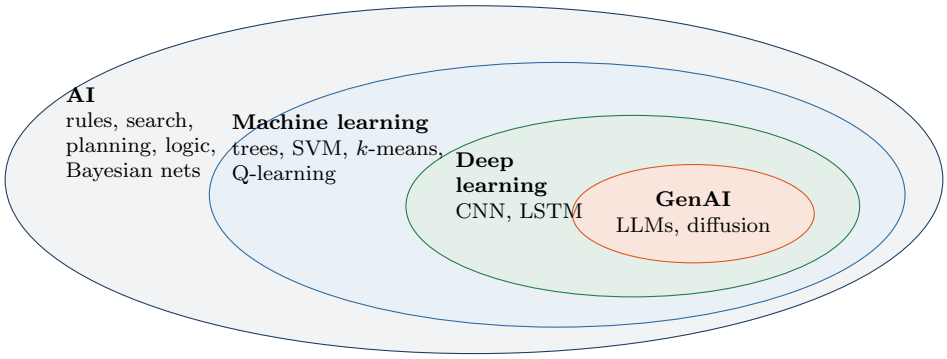


Figure B3: the nested hierarchy. Generative AI is built from deep networks.

Data needs grow along the hierarchy: classical AI needs no training data; shallow ML typically 1,000–100,000 labelled examples; deep learning often millions; large language models are trained on trillions of tokens. With 200 labelled examples, a heavily regularised shallow model or a rule system is usually wiser than a deep network.

18.1 What it means for a machine to learn

Definition — learning (Mitchell, 1997)

A computer program is said to **learn** from experience E with respect to some class of tasks T and performance measure P if its performance at tasks in T, as measured by P, improves with experience E.

The definition is useful because it forces all three elements to be named. For a radar classifier, T is "assign a class to each track", P is per-class recall at an agreed false-alarm rate, and E is a set of labelled historical tracks. For a wargame agent, T is "choose moves", P is the win rate against a stated opponent, and E is the agent's own simulated games. A claim that "the system learns" without T, P and E is not testable.

18.2 The vocabulary of data

- An **example** (instance, sample, record) is one object: a radar track, an image chip, a log line, a game state.
- A **feature** (attribute, input variable) is one measured property of an example: pulse repetition frequency, altitude, pixel intensity. An example with d features is a vector $x \in \mathbb{R}^d$; a dataset of N examples is a matrix $X \in \mathbb{R}^{N \times d}$.
- A **label** (target, output) y is the quantity to be predicted: a class ("bomber") or a number (mission duration).
- The **feature space** is the d-dimensional space in which every example is a point. Learning is the search for structure in this space: boundaries between classes, clusters, low-dimensional surfaces.
- Data come in recognisable **modalities**: tabular (rows and columns), images (grids of pixels), sequences and time series (radar tracks, logs), text, graphs (communication networks), and point sets. Model families have grown up around each modality, as Section 25 shows.

18.3 Hypotheses, inductive bias and generalisation

A learning algorithm chooses a function $\hat{f}$ from a **hypothesis space** $\mathcal{H}$ — the set of functions it is able to represent. Logistic regression's $\mathcal{H}$ is all linear decision boundaries; a decision tree's is all axis-aligned box partitions; a neural network's is very large. The assumptions that make one hypothesis preferable to another before seeing any data are its **inductive bias**: "the boundary is linear", "nearby points have the same label", "simpler explanations are better".

Generalisation is the ability to perform well on new examples drawn from the same distribution as the training data. It is the only thing that matters: a model that is perfect on its training data and poor on new data is useless. The difference between training error and error on new data is the **generalisation gap**.

Key point

No free lunch. Averaged over all possible problems, no learning algorithm is better than any other (Wolpert, 1996). An algorithm wins only because its inductive bias matches the structure of the problem at hand. This is why the course teaches many methods and insists on testing them on the deployment domain, rather than declaring one "best".

18.4 Parametric and non-parametric; discriminative and generative

Parametric models summarise the data in a fixed number of parameters (logistic regression with $d + 1$ weights); they are compact and fast but limited by their form. **Non-parametric** models grow with the data (k-nearest neighbours stores every example; a fully grown tree can have as many leaves as examples); they are flexible but slower and data-hungry.

Discriminative models learn $P(y \mid x)$ or a decision boundary directly (logistic regression, SVM). **Generative** models learn how the data are produced, $P(x \mid y)P(y)$ or simply $P(x)$ (Naive Bayes, Gaussian mixtures, large language models). Generative models can create new samples and handle missing inputs; discriminative models usually classify better when data are plentiful.

18.5 The machine-learning workflow

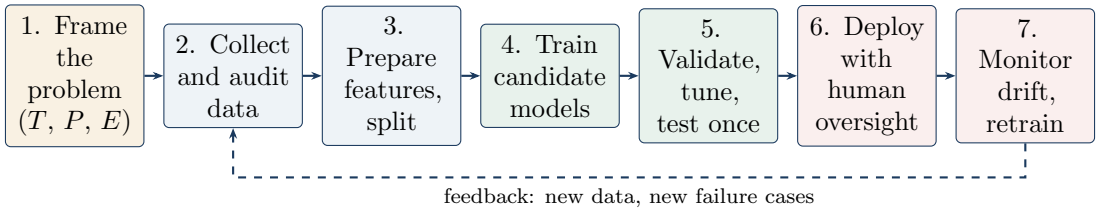


Figure B4: the machine-learning life cycle. Steps 1, 2 and 7 decide success more often than step 4.

Practitioners consistently report that most effort goes into steps 1–3. Part A of this text is, in effect, a manual for step 2: auditing data for selection, confounding, base rates and missingness. The choice of algorithm in step 4 usually matters less than the quality of the data and the honesty of the evaluation in step 5.

19 The Five Machine-Learning Paradigms

Paradigm	Labels	What is learned; defence example
Supervised	all	mapping $X \to Y$; radar threat classification
Unsupervised	none	structure (clusters, densities); OPFOR movement archetypes
Semi-supervised	few (∼5%)	labels propagated through structure; SAR vehicle recognition with 200 labelled and 20,000 unlabelled images
Self-supervised	derived from data	representations via pretext tasks; pre-training on 500,000 raw radar sweeps
Reinforcement	reward only	a policy from interaction; Blue-Force agent in a wargame simulator

The three master questions organise the whole course: (1) Do I have labels? (2) Is the problem interactive, with sequential decisions? (3) Are there multiple agents?

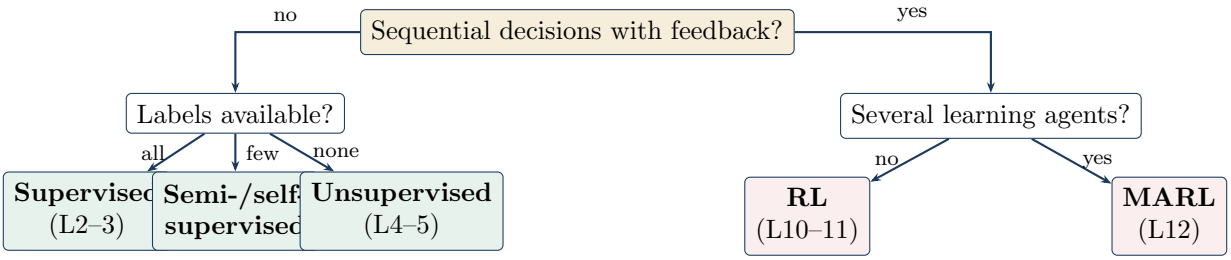


Figure B5: choosing a paradigm. Lecture numbers show where each is developed. Neural networks (L6–9) are not a separate paradigm: they are the function approximators used inside all of them.

The paradigms are not exclusive. A modern system often combines them: self-supervised pre-training on unlabelled sensor data, supervised fine-tuning on a few labelled examples, unsupervised anomaly detection to flag novelties, and reinforcement learning for the decision layer.

20 Supervised Learning

20.1 Formulation

Given a training set $D = \{(x_i, y_i)\}_{i=1}^{N}$ with $x_i \in \mathbb{R}^d$, supervised learning seeks $f : \mathbb{R}^d \to Y$ so that $f(x_{\text{new}}) \approx y_{\text{true}}$ for examples it has never seen. **Classification** has a finite set of labels Y (threat classes); **regression** has $Y \subseteq \mathbb{R}$ (mission duration, fuel required, casualty estimate). The examples are assumed to be drawn independently from one fixed distribution — the *i.i.d. assumption* — and the whole edifice of evaluation rests on it. When deployment data come from a different distribution (Section 28.3), every guarantee lapses.

20.2 Loss functions and empirical risk

A **loss function** $\ell(\hat{y}, y)$ measures the cost of predicting $\hat{y}$ when the truth is y. Learning is **empirical risk minimisation**: choose the parameters θ that minimise the average training loss, usually plus a penalty $R(\theta)$ on complexity:

$$\theta^{*} = \arg\min_{\theta} \Big[\underbrace{\tfrac{1}{N} \textstyle\sum_{i=1}^{N} \ell\big(f(x_i;\theta), y_i\big)}_{\text{empirical risk}} + \lambda R(\theta) \Big].$$

Loss	Formula	Used by; behaviour
Squared	$(y - \hat{y})^2$	linear regression, neural nets; punishes large errors heavily, sensitive to outliers
Absolute	$\lvert y - \hat{y} \rvert$	robust regression; median-seeking, tolerant of outliers
Huber	quadratic if $\lvert e \rvert \le \delta$, linear beyond	robust regression; compromise of the two
0–1	$\mathbf{1}[\hat{y} \neq y]$	the error rate itself; not differentiable, so used for evaluation not training
Cross-entropy	$-\sum_k y_k \log \hat{p}_k$	logistic regression, neural classifiers; punishes confident wrong answers severely
Hinge	$\max(0, 1 - y\,s)$, $y \in \{\pm 1\}$	support vector machines; zero once the margin is satisfied

The choice of loss is a modelling decision with operational meaning. Squared error says a 4-hour error is four times as bad as a 2-hour one; in logistics planning an under-estimate (running out of fuel) may be far worse than an over-estimate, which calls for an asymmetric loss (Lecture 3, quantile regression).

20.3 Gradient descent

Most models have no closed-form minimiser, so they are trained by **gradient descent**: repeatedly move the parameters a small step against the gradient of the loss,

$$\theta \leftarrow \theta - \alpha \, \nabla_{\theta} \mathcal{L}(\theta),$$

where α is the **learning rate**. The gradient points uphill; its negative points to the steepest descent. Too large an α overshoots and diverges; too small an α crawls. **Stochastic gradient descent** (SGD) estimates the gradient from a random *mini-batch* of examples, which is cheaper per step and adds useful noise.

Worked example — fitting a line by gradient descent, step by step

Three sorties give EW intensity x and detection delay y (s): $(1,2)$, $(2,4)$, $(3,5)$. Model $\hat{y} = wx + b$, loss $\mathcal{L} = \frac{1}{3}\sum(y_i - \hat{y}_i)^2$. The gradients are

$$\frac{\partial \mathcal{L}}{\partial w} = -\frac{2}{3}\sum x_i(y_i - \hat{y}_i), \qquad \frac{\partial \mathcal{L}}{\partial b} = -\frac{2}{3}\sum (y_i - \hat{y}_i).$$

Start at $w = b = 0$ with $\alpha = 0.1$.

Iteration 1. Predictions $0, 0, 0$; residuals $2, 4, 5$; $\mathcal{L} = (4 + 16 + 25)/3 = 15.0$. $\partial_w = -\frac{2}{3}(2 + 8 + 15) = -16.67$; $\partial_b = -\frac{2}{3}(11) = -7.33$. Update: $w = 1.667$, $b = 0.733$.
Iteration 2. Predictions $2.400, 4.067, 5.733$; residuals $-0.400, -0.067, -0.733$; $\mathcal{L} = 0.234$. $\partial_w = 1.822$, $\partial_b = 0.800$. Update: $w = 1.484$, $b = 0.653$.
Iteration 3. $\mathcal{L} = 0.058$; update to $w = 1.504$, $b = 0.662$.

The exact least-squares solution is $w = 1.5$, $b = 0.667$ with minimum loss 0.0556: three steps have nearly reached it. The loss fell from 15 to 0.058 because each step followed the slope downhill.

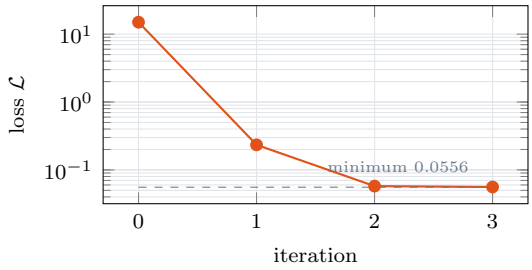


Figure B6: loss against iteration for the worked example (log scale).

20.4 Three classic learners, worked by hand

20.4.1 Logistic regression: a probability from a weighted sum

Logistic regression computes a score $z = w^\top x + b$ and squashes it into a probability with the **sigmoid** $\sigma(z) = 1/(1 + e^{-z})$. It is trained by minimising cross-entropy; the gradient for one example is remarkably simple: $\nabla_w \ell = (\hat{p} - y)\,x$ and $\partial_b \ell = \hat{p} - y$.

Worked example — one prediction and one learning step

A track has normalised speed $x_1 = 1.2$ and EW activity $x_2 = 1$. Current weights $w = (0.8,\ 1.5)$, $b = -1.0$. Then $z = 0.8(1.2) + 1.5(1) - 1.0 = 1.46$ and $\hat{p} = \sigma(1.46) = 0.812$: the model says 81% hostile. The track is in fact hostile ($y = 1$), so the loss is $-\ln 0.812 = 0.209$.
Gradient: $(\hat{p} - y)x = (-0.188)(1.2,\ 1) = (-0.226,\ -0.188)$; $\partial_b = -0.188$. With $\alpha = 0.5$: $w = (0.913,\ 1.594)$, $b = -0.906$. The new prediction for this track is $\hat{p} = 0.856$. Learning nudged the weights of the features that were present in the direction that would have made the right answer more likely.

Each weight has a direct interpretation: increasing x_j by one unit multiplies the odds $\hat{p}/(1 - \hat{p})$ by e^{w_j}. That interpretability is why logistic regression remains a standard baseline in defence analysis.

20.4.2 k-nearest neighbours: judged by the company you keep

k-NN stores the training set. To classify a new point it finds the k closest training points (usually by Euclidean distance on standardised features) and takes a majority vote. It has no training

phase and a natural inductive bias — nearby points share labels — but prediction is slow for large data and it is ruined by unscaled features (a feature measured in metres swamps one measured in kilometres) and by irrelevant features. Lecture 2 develops it.

20.4.3 Decision trees: twenty questions with data

A decision tree asks a sequence of yes/no questions about features, each chosen to make the resulting groups as pure as possible. Purity is measured by **Gini impurity** $G = 1 - \sum_k p_k^2$ or **entropy** $H = -\sum_k p_k \log_2 p_k$.

Worked example — choosing a split

A node holds 10 tracks: 6 hostile, 4 friendly. $G = 1 - (0.6^2 + 0.4^2) = 0.48$; $H = 0.971$ bits. Splitting on "EW active?" gives a left child with 5 hostile and 0 friendly ($G = 0$, $H = 0$) and a right child with 1 hostile and 4 friendly ($G = 1 - (0.2^2 + 0.8^2) = 0.32$, $H = 0.722$). Weighted child impurity: $G = 0.5(0) + 0.5(0.32) = 0.16$, a decrease of 0.32; information gain $= 0.971 - 0.5(0.722) = 0.610$ bits. The tree compares this with every other candidate split and takes the best.

Trees are readable ("IF EW active THEN hostile") but a deep tree memorises noise. Averaging many randomised trees — a **random forest** — fixes this and is one of the strongest tabular methods (Lecture 2).

20.5 Underfitting, overfitting and model capacity

A model's **capacity** is the richness of the functions it can represent. Too little capacity **underfits**: the model misses real structure and errs on training and test data alike (high bias). Too much capacity **overfits**: it fits the noise of the particular training sample and fails on new data (high variance). The classical picture is a U-shaped test-error curve:

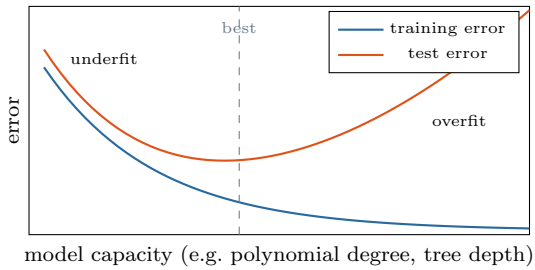


Figure B7: training error always falls with capacity; test error falls, then rises.

The remedies for overfitting are: more (and more varied) data; **regularisation** that penalises complexity — the L2 penalty $\lambda \sum w_j^2$ (ridge) shrinks weights, the L1 penalty $\lambda \sum |w_j|$ (lasso) sets some exactly to zero; early stopping; and ensembles. The remedy for underfitting is a richer model or better features. Section 28.2 gives the formal bias–variance decomposition and the mine-detector case.

20.6 Evaluation protocol

Split the data into training (e.g. 70%), validation (15%) and test (15%) sets. Train on the training set; tune hyper-parameters (tree depth, k, λ, learning rate) on the validation set or by **k-fold cross-validation**, in which the data are divided into k folds and each fold serves once as validation while the other $k-1$ train; report the mean and spread across folds. Touch the test set *once*, at the end. Three rules prevent the commonest errors:

1. **Split before anything else.** Scaling, feature selection and oversampling are fitted on the training fold only. Fitting them on all the data leaks test information into training.
2. **Split by the unit of independence.** Radar returns from one track, or log lines from one day, are correlated. Put whole tracks, sorties or days into one side of the split, never both.
3. **Stratify.** Keep class proportions equal across folds so that rare classes appear in every fold.

20.7 Metrics from the confusion matrix

For a binary detector with counts TP (threats correctly flagged), FP (false alarms), FN (missed threats) and TN:

$$\text{accuracy} = \frac{TP+TN}{\text{all}}, \qquad \text{precision} = \frac{TP}{TP+FP}, \qquad \text{recall (TPR)} = \frac{TP}{TP+FN},$$

$$\text{FPR} = \frac{FP}{FP+TN}, \qquad F_1 = \frac{2PR}{P+R}.$$

Precision answers "of my alarms, how many were real?"; recall answers "of the real threats, how many did I catch?". A classifier outputs a score; the **threshold** turns scores into decisions, and moving it trades recall against false alarms.

Worked example — building an ROC curve from ten scored tracks

Scores (descending) with true labels (1 = threat): 0.95(1), 0.90(1), 0.80(0), 0.70(1), 0.60(1), 0.55(0), 0.40(0), 0.30(1), 0.20(0), 0.10(0). There are 5 threats and 5 non-threats.

Threshold	TP	FP	TPR	FPR (precision)
0.85	2	0	0.4	0.0 (1.00)
0.65	3	1	0.6	0.2 (0.75)
0.50	4	2	0.8	0.4 (0.67)
0.35	4	3	0.8	0.6 (0.57)
0.00	5	5	1.0	1.0 (0.50)

Plotting TPR against FPR for every threshold gives the ROC curve. The area under it (AUC) equals the fraction of (threat, non-threat) pairs in which the threat has the higher score: here 20 of 25 pairs, so AUC = 0.80. To catch every threat (TPR = 1) the operator must accept that half the alarms are false.

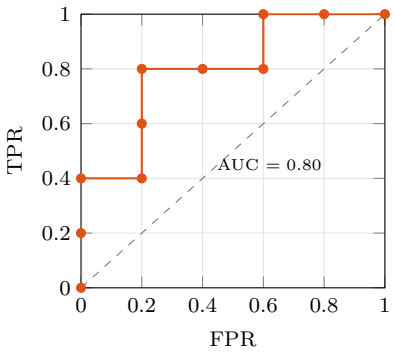


Figure B8: the ROC staircase for the ten tracks; the diagonal is random guessing.

For rare threats, the precision–recall view is more honest than ROC (Part A, the base-rate fallacy; Lecture 2).

20.8 Case study: radar threat classification

Five classes (friendly fighter, adversary bomber, missile, UAV, noise), N = 12,500, five radar features (peak frequency, pulse width, PRF, amplitude, scan rate), 80/20 split. Test accuracies: logistic regression 79.2%, SVM 87.3%, random forest 93.1%, 1D-CNN 97.2% (F1 0.971, 2.1 ms inference). Even the best model labels some enemy bombers as friendly — a false negative on the threat class, the costliest error. Model selection in defence must prioritise **recall on the threat class**, with cost-weighted thresholds, rather than overall accuracy.

Pitfall — internal consistency of reported results

The lecture's confusion matrix has a diagonal of $493+488+487+489+496 = 2{,}453$ correct out of 2,500, i.e. 98.1%, whereas the headline figure quoted is 97.2%; the text also mentions three bombers labelled friendly where the matrix shows two. Inconsistencies like these are common in reports. Always recompute headline numbers from the underlying tables before relying on them.

20.9 A map of supervised algorithms

Family	Inductive bias	Strengths / weaknesses	Lecture
Linear / logistic	boundary or trend is linear	interpretable, fast / misses interactions	2, 3
k-NN	neighbours share labels	no training / slow, needs scaling	2
Decision tree	axis-aligned rules	readable / unstable, overfits	2
Random forest, boosting	averages or sequences of trees	accurate on tables / less transparent	2, 3
SVM, SVR	maximum margin, kernels	strong with few examples / tuning, scaling	2, 3
Naive Bayes	features independent given class	very fast / over-confident	2
Neural networks	hierarchical features	images, signals, text / data-hungry, opaque	1, 2
Kalman filter	linear-Gaussian dynamics	principled tracking and fusion / model needed	3

Check your understanding — supervised learning

(1) Why may the test set be used only once? (2) A model scores 99% training and 71% test accuracy: diagnose and suggest two remedies. (3) In the ROC example, which threshold would you choose if a missed threat costs ten times a false alarm?

Answers. (1) Every look at test results that influences a decision turns the test set into a validation set and biases the estimate upward. (2) Overfitting (high variance): add regularisation or reduce depth/capacity; add data or augmentation; check for leakage between splits. (3) Total cost = $10\,FN + FP$: threshold 0.85 gives $10(3)+0 = 30$; 0.65 gives 21; 0.50 gives 12; 0.35 gives 13; a threshold of 0.25 (just below the lowest-scored threat) gives $10(0)+3 = 3$; flagging everything gives 5. Choose 0.25. With a heavy cost on misses the optimal threshold moves down until every threat is caught — and a second, confirmatory sensor is then needed to filter the extra false alarms.

21 Unsupervised Learning

Unsupervised learning receives only inputs $x_1, \ldots, x_N$ — no labels — and looks for structure. It answers four kinds of question:

- **Clustering:** which examples belong together? (OPFOR manoeuvre archetypes; emitter families in SIGINT.)
- **Density estimation:** how probable is this example? (The basis of anomaly detection, Lecture 5.)
- **Dimensionality reduction:** can 50 features be summarised by 3 without losing much? (Visualisation, compression, noise removal.)
- **Anomaly detection:** which examples do not fit? (Intrusions, unusual force behaviour.)

Unsupervised learning matters disproportionately in defence because labels are expensive (an analyst-hour per image), slow (after-action reports arrive weeks later) and incomplete (nobody can label a tactic that has never been seen). Its outputs, however, are *hypotheses*: an algorithm will always return clusters, even from pure noise. Validation by independent methods and by domain experts is part of the method, not an optional extra.

21.1 Distance: the hidden decision

Clustering depends entirely on what "close" means. For $x = (3, 4)$ and $z = (0, 0)$: Euclidean distance $\sqrt{9 + 16} = 5$; Manhattan distance $3 + 4 = 7$; Chebyshev distance $\max(3, 4) = 4$. **Cosine similarity** $x^\top z / (\|x\| \|z\|)$ compares directions and ignores magnitude — natural for text and spectra. **Mahalanobis distance** $\sqrt{(x - z)^\top \Sigma^{-1} (x - z)}$ accounts for the spread and correlation of features (Lecture 5).

Pitfall — unscaled features

Trajectories described by speed in m/s (range 0–300) and heading change in radians (range 0–3) are clustered almost entirely by speed, because speed differences dominate the Euclidean distance. Standardise each feature to zero mean and unit variance ($z = (x - \mu)/\sigma$) unless there is a reason not to — and record that choice, because it changes the clusters.

21.2 k-Means in detail

k-Means represents each cluster by its centroid μ_j and minimises the within-cluster sum of squares

$$J = \sum_{j=1}^{k} \sum_{x \in C_j} \|x - \mu_j\|^2.$$

Algorithm — Lloyd's k-means

1. Choose k and initial centroids $\mu_1, \ldots, \mu_k$ (better: k-means++, which spreads them out).
2. **Assign:** put each point in the cluster of its nearest centroid.
3. **Update:** move each centroid to the mean of its assigned points.
4. Repeat 2–3 until no assignment changes.

Neither step can increase J — assignment picks the nearest centroid for each point, and the mean is the point that minimises squared distance to a set — and there are finitely many partitions, so

the algorithm always terminates. It may terminate in a *local* minimum, so it is run from several starts and the lowest-J result kept.

Worked example — k-means with a poor start

Seven contacts on a 2-D map: $A(1,1)$, $B(1.5,2)$, $C(3,3.5)$, $D(5,7)$, $E(3.5,5)$, $F(4.5,5)$, $G(3.5,4.5)$; $k=2$, initial centroids deliberately placed badly at A and B.

Round 1. Every point except A is nearer B (e.g. C: 3.20 to A, 2.12 to B). Clusters $\{A\}$ and $\{B,\dots,G\}$; new centroids $(1,1)$ and $(3.5,4.5)$; $J=21.5$.

Round 2. B is now 1.12 from $(1,1)$ but 3.20 from $(3.5,4.5)$, so it migrates. Clusters $\{A,B\}$ and $\{C,D,E,F,G\}$; centroids $(1.25,1.5)$ and $(3.9,5.0)$; $J=9.825$.

Round 3. No point changes cluster: converged. The bad start cost one extra round here; in harder data it can trap the algorithm in a poor solution, which is why k-means++ and multiple restarts are standard.

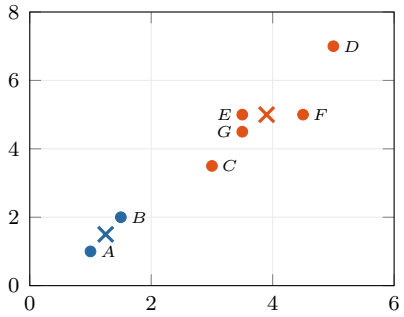


Figure B9: final clusters; crosses mark centroids.

Assumptions to remember. k-means favours compact, roughly spherical clusters of similar size, needs k in advance, and assigns every point — including outliers, which drag centroids. DBSCAN (density), Gaussian mixtures (elliptical, soft) and hierarchical clustering (a whole tree of nested clusters) relax these assumptions; Lecture 4 compares them on 8,000 OPFOR trajectories.

21.3 How many clusters? Elbow and silhouette

WCSS (J) always falls as k grows — with $k=N$ it is zero — so the minimum J cannot choose k. The **elbow method** plots J against k and looks for the point where extra clusters stop paying. The **silhouette** of a point i compares its mean distance $a(i)$ to the other members of its own cluster with its mean distance $b(i)$ to the members of the nearest other cluster:

$$s(i)=\frac{b(i)-a(i)}{\max\{a(i),b(i)\}}\in[-1,1].$$

Values near 1 mean well inside its cluster, near 0 on a border, negative probably misassigned. For contact C above: $a=2.21$ (mean distance to D,E,F,G) and $b=2.66$ (mean distance to A,B), so $s=(2.66-2.21)/2.66=0.17$ — a borderline point, which matches its position between the groups. The average silhouette over all points summarises a clustering; above about 0.7 indicates strong structure.

21.4 Soft clustering: Gaussian mixtures

A Gaussian mixture model (GMM) assumes each point was generated by one of k Gaussian components with weights π_j, means μ_j and covariances Σ_j. Instead of a hard label it gives each point a **responsibility** $r_{ij} = P(\text{component } j \mid x_i)$, computed by Bayes' rule:

$$r_{ij} = \frac{\pi_j \, \mathcal{N}(x_i; \mu_j, \Sigma_j)}{\sum_l \pi_l \, \mathcal{N}(x_i; \mu_l, \Sigma_l)}.$$

Worked example — responsibilities in one dimension

Two equally weighted components: $\mathcal{N}(1, 1^2)$ ("patrol" speeds) and $\mathcal{N}(5, 1.5^2)$ ("transit" speeds). A unit moving at 2 has responsibility 0.87 for patrol; at 3, only 0.33; at 4, 0.02. The point at 3 is ambiguous and the model says so — useful information that hard clustering throws away.

The **EM algorithm** fits a GMM by alternating an E-step (compute responsibilities) and an M-step (re-estimate weights, means and covariances as responsibility-weighted averages). k-means is the special case with equal spherical covariances shrinking to zero.

21.5 Dimensionality reduction: PCA

Principal component analysis finds orthogonal directions of maximum variance. Centre the data, form the covariance matrix S, and take its eigenvectors; the eigenvalue λ_j is the variance along eigenvector j, and $\lambda_j / \sum_l \lambda_l$ is the fraction of variance it explains.

Worked example — PCA of ten two-feature observations

Ten readings of two correlated sensor channels have means $(1.81, 1.91)$ and covariance $S = \begin{pmatrix} 0.617 & 0.615 \\ 0.615 & 0.717 \end{pmatrix}$. Its eigenvalues are 1.284 and 0.049, with first eigenvector $(0.678, 0.735)$. The first principal component explains $1.284/1.333 = 96.3\%$ of the variance: the two channels essentially measure one underlying quantity, and a single number $z = 0.678(x_1 - 1.81) + 0.735(x_2 - 1.91)$ summarises each reading with little loss.

PCA is linear. Non-linear methods (t-SNE, UMAP, autoencoders) unfold curved structure and are valuable for visualisation, but they distort distances and cluster sizes; use them to look, not to measure (Lecture 4).

21.6 Case study: OPFOR tactic clustering

In the lecture's OPFOR example ($N = 4{,}800$ trajectories) the elbow and a silhouette of 0.683 supported $k = 4$ archetypes (patrol, assault, transit, static defence), and the assault cluster's high direction-change rate revealed a flanking signature that threshold-based analysts had missed. Lecture 4 treats clustering in depth and shows DBSCAN isolating deception manoeuvres as "noise".

22 Semi-supervised Learning

When labels are scarce but unlabelled data are plentiful — 200 analyst-labelled SAR chips and 20,000 unlabelled ones — semi-supervised learning uses the unlabelled data to shape the decision boundary. It works only if the unlabelled data carry information about the labels, which requires at least one of three assumptions:

- **Smoothness:** points close together in a high-density region have the same label.

- **Cluster:** points in the same cluster share a label, so the boundary should pass through low-density gaps.
- **Manifold:** the data lie near a low-dimensional surface, and distances along it are more meaningful than straight-line distances.

22.1 Self-training (pseudo-labelling)

Algorithm — self-training

1. Train a model on the labelled set L.
2. Predict the unlabelled set U; select predictions with confidence above a threshold τ (e.g. 0.95).
3. Add those examples with their predicted ("pseudo") labels to L; remove them from U.
4. Retrain and repeat until no confident predictions remain or validation accuracy stops improving.

The danger is **confirmation bias**: early mistakes are added as if they were truth and then reinforced. A high threshold, class-balanced selection and a clean labelled validation set limit the damage.

22.2 Label propagation

Build a graph whose nodes are all examples and whose edges join similar examples. Labelled nodes keep their labels; every unlabelled node repeatedly takes the weighted average of its neighbours' label scores until the values settle. On a chain of five contacts in which contact 1 is known hostile (score 1) and contact 5 known friendly (score 0), propagation converges to $1, 0.75, 0.5, 0.25, 0$: each unknown contact is labelled by how strongly it is connected to each known one, and contact 3 is honestly undecided.

22.3 Modern consistency methods

FixMatch and its relatives combine pseudo-labelling with **consistency regularisation**: the model must give the same prediction for a weakly augmented and a strongly augmented version of the same unlabelled image. In the lecture's SAR example, 200 labels alone gave 71.2% accuracy, adding 20,000 unlabelled images gave 88.4%, and full labelling of 20,200 images gave 94.1% — most of the benefit of labelling, for 1% of the labelling effort.

23 Self-supervised Learning

Self-supervised learning manufactures a supervised task from unlabelled data by hiding part of each example and asking the model to predict it. The task itself is not the goal; the **representation** the network learns while solving it is. That representation is then reused for the real (downstream) task with few labels.

23.1 Pretext tasks

- **Masked modelling:** hide 15% of words (BERT) or 75% of image patches (masked autoencoders) and reconstruct them. Large language models use the extreme version: predict every next token.

- **Contrastive learning:** create two random augmentations (crop, rotate, add noise) of each example; train the network so that the two views of the same example are close in representation space and views of different examples are far apart (SimCLR, MoCo).
- **Prediction of transformations or the future:** predict the rotation applied to an image, or the next segment of a radar track.

23.2 The contrastive loss

For an anchor view with representation z, its positive partner z^+ and negatives $z_1^-, \ldots$, the InfoNCE loss is

$$\mathcal{L} = -\log \frac{\exp(\mathrm{sim}(z, z^+)/\tau)}{\exp(\mathrm{sim}(z, z^+)/\tau) + \sum_k \exp(\mathrm{sim}(z, z_k^-)/\tau)},$$

where sim is cosine similarity and τ a temperature. It is the cross-entropy of picking the positive out of the crowd.

Worked example — one contrastive term

Similarities: positive 0.9; negatives 0.2, 0.1, -0.3. With $\tau = 0.5$ the softmax probability of the positive is 0.650 and the loss $-\ln 0.650 = 0.431$. With $\tau = 0.1$ the probability is 0.999 and the loss 0.001. A low temperature sharpens the comparison and focuses learning on the hardest negatives.

23.3 Using the representation

After pre-training, either freeze the network and train a small classifier on top (a **linear probe**, which also measures representation quality) or **fine-tune** the whole network on the labelled data with a small learning rate. Pre-training on 500,000 unlabelled radar signals and fine-tuning on 1,000 labelled ones raised accuracy from 79.2% to 93.7% in the lecture's example. Very large self-supervised models adapted to many tasks are called **foundation models**.

Pitfall — inherited bias

A representation learned from unlabelled data inherits whatever is over- or under-represented in that data. Pre-training on peacetime emissions may encode peacetime patterns that wartime emitters deliberately break.

24 Reinforcement Learning

Supervised learning is told the right answer for every example. Reinforcement learning (RL) is told only how good the *consequences* of its behaviour were, often long after the decisive action. It is the paradigm for sequential decision-making: manoeuvring units, routing drones, allocating sensors, playing a wargame. This section builds the core ideas from first principles on a small example that can be solved by hand; Lectures 10–12 extend them to deep networks, continuous control and teams of agents.

24.1 The agent–environment loop

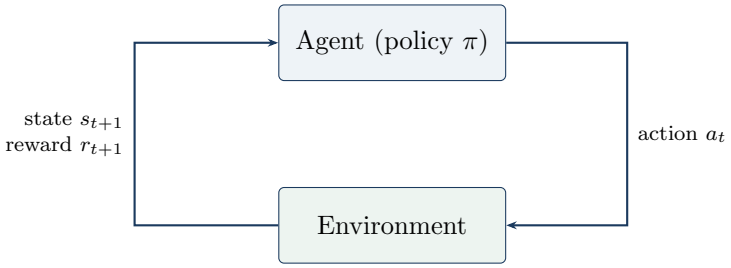


Figure B10: at each step the agent observes the state, acts, and receives a reward and the next state.

At time t the agent observes state s_t, chooses action a_t, and the environment returns a reward r_{t+1} and a new state s_{t+1}. The interaction produces a trajectory $s_0, a_0, r_1, s_1, a_1, r_2, \ldots$ that ends at a terminal state (an *episode*) or continues indefinitely. Three features make RL harder than supervised learning: the feedback is **evaluative** (how good, not what was right), it is **delayed** (credit must be assigned to earlier actions), and the data depend on the agent's own behaviour (it must **explore** to learn about actions it has not tried).

24.2 Markov decision processes

Definition — Markov decision process (MDP)

An MDP is a tuple (S, A, P, R, γ): a set of states S; a set of actions A; transition probabilities $P(s' \mid s, a)$; a reward function $R(s, a, s')$; and a discount factor $\gamma \in [0, 1)$. The **Markov property** requires that the next state and reward depend only on the current state and action, not on the earlier history: the state must summarise everything relevant.

Worked example — the patrol grid (used throughout this section)

A unit moves on a 2×3 grid. The top-right cell S3 is the objective (reward $+10$, episode ends); the bottom-right cell S6 is a minefield (reward -10, episode ends). Every other move costs -1 (time and fuel). Actions are Up, Down, Left, Right; moving into the edge leaves the unit where it is. The unit starts in S4. Transitions are deterministic and $\gamma = 0.9$.

S1	S2	S3 goal +10
S4 start	S5	S6 mines −10

There are four non-terminal states and four actions, so the whole problem is described by 16 state–action pairs.

24.3 Return and discounting

The agent maximises the **return**, the discounted sum of future rewards:

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}.$$

Discounting keeps infinite sums finite, expresses a preference for sooner rewards, and sets an effective planning horizon of about $1/(1-\gamma)$ steps (10 steps for $\gamma = 0.9$, 100 for 0.99).

Worked example — return of a route

From S4 the route Up, Right, Right (S4→S1→S2→S3) earns $-1, -1, +10$, so $G = -1 + 0.9(-1) + 0.9^2(10) = -1 - 0.9 + 8.1 = 6.2$. The route Right, Right runs into the minefield: $G = -1 + 0.9(-10) = -10.0$. The route Right, Up, Right is as good as the first: $-1 - 0.9 + 8.1 = 6.2$.

24.4 Policies and value functions

A **policy** π says what to do in each state — deterministically, $a = \pi(s)$, or stochastically, $\pi(a \mid s)$. Two functions measure how good a policy is:

$$V^{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid s_t = s], \qquad Q^{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid s_t = s, a_t = a].$$

$V^{\pi}(s)$ is the expected return from state s following π; $Q^{\pi}(s, a)$ is the expected return from taking action a in s and following π afterwards. The best achievable values are $V^*(s) = \max_{\pi} V^{\pi}(s)$ and $Q^*(s, a) = \max_{\pi} Q^{\pi}(s, a)$. If we know Q^*, acting optimally is trivial: in each state pick the action with the largest Q^*,

$$\pi^*(s) = \arg\max_{a} Q^*(s, a).$$

24.5 The Q-table

When there are few states and actions, Q can be stored as a **table** with one row per state and one column per action. The table *is* the agent's knowledge: reading a row tells it how good each action is from that state; the largest entry in the row is the action it will take.

Worked example — the optimal Q-table of the patrol grid

State	Up	Down	Left	Right	Best action
S1	6.20	4.58	6.20	**8.00**	Right
S2	8.00	6.20	6.20	**10.00**	Right
S4	**6.20**	4.58	4.58	**6.20**	Up or Right
S5	**8.00**	6.20	4.58	−10.00	Up

How to read an entry: $Q^*(\text{S5}, \text{Up}) = 8.00$ because moving up costs -1 and lands in S2, from which the best one can do is 10: $-1 + 0.9 \times 10 = 8.0$. $Q^*(\text{S1}, \text{Down}) = 4.58$ because the move costs -1 and lands in S4, worth 6.2: $-1 + 0.9 \times 6.2 = 4.58$. $Q^*(\text{S5}, \text{Right}) = -10$: the minefield. The greedy policy read off the table avoids the mines and reaches the objective in three moves.

24.6 The Bellman equations

The values in the table are not arbitrary: each is consistent with its neighbours. Splitting the return into the first reward plus the discounted rest gives the **Bellman equations**:

$$V^{\pi}(s) = \sum_{a} \pi(a \mid s) \sum_{s'} P(s' \mid s, a)\big[R(s, a, s') + \gamma V^{\pi}(s')\big],$$

$$Q^*(s, a) = \sum_{s'} P(s' \mid s, a)\Big[R(s, a, s') + \gamma \max_{a'} Q^*(s', a')\Big].$$

The second, the **Bellman optimality equation**, says: the value of an action is its immediate reward plus the discounted value of acting optimally from wherever it leads. It is exactly the calculation used to explain the table entries above.

24.6.1 Solving with a model: value iteration

If the transition probabilities and rewards are known, the Bellman optimality equation can be turned into an update and applied repeatedly to every state–action pair:

$$Q_{k+1}(s,a) \leftarrow \sum_{s'} P(s' \mid s,a)\big[R + \gamma \max_{a'} Q_k(s',a')\big].$$

Starting from $Q_0 = 0$, the first sweep learns only immediate rewards ($Q_1(\mathrm{S2},\mathrm{Right}) = 10$, $Q_1(\mathrm{S5},\mathrm{Right}) = -10$, other entries -1); the second sweep propagates the goal one step further back ($Q_2(\mathrm{S1},\mathrm{Right}) = -1 + 0.9 \times 10 = 8$); within a few sweeps the table above is reached. Because the update is a contraction by a factor γ, it converges to Q^* from any start. This is **dynamic programming**: planning with a known model.

24.6.2 Learning without a model

In a real engagement or a complex simulator we do not have P and R as formulas; we can only try actions and observe what happens. **Model-free** RL learns values from sampled experience. There are two ways to form a learning target from experience:

- **Monte Carlo:** wait until the episode ends and use the actual return G_t. Unbiased but noisy, and it cannot learn until the episode finishes.
- **Temporal difference (TD):** after one step, use $r_{t+1} + \gamma \hat{V}(s_{t+1})$, where $\hat{V}$ is the current estimate of the next state's value. This *bootstraps* from the agent's own estimates, so it learns step by step with lower variance, at the cost of some bias while the estimates are still wrong.

24.7 Q-learning

Q-learning (Watkins, 1989) is the TD method for Q^*. After each step (s,a,r,s') it nudges the table entry toward the Bellman target:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\Big[\underbrace{r + \gamma \max_{a'} Q(s',a')}_{\text{TD target}} - Q(s,a)\Big],$$

where $\alpha \in (0,1]$ is the learning rate and the bracket is the **TD error**. If s' is terminal, the target is just r.

Algorithm — tabular Q-learning

1. Initialise $Q(s,a) = 0$ for all non-terminal s and all a.
2. For each episode: set s to the start state. Repeat until s is terminal:
   (a) choose a by ε-greedy: with probability ε a random action, otherwise $\arg\max_a Q(s,a)$;
   (b) take a; observe r and s';
   (c) $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$;
   (d) $s \leftarrow s'$.

Worked example — four episodes of Q-learning on the patrol grid ($\alpha = 0.5$, $\gamma = 0.9$)

All entries start at 0. The actions below are what the exploring agent happened to choose.
Episode 1. S4 Right → S5, $r = -1$: target $-1 + 0.9 \max_a Q(\mathrm{S5},a) = -1 + 0 = -1$; $Q(\mathrm{S4},\mathrm{R}) = 0 + 0.5(-1 - 0) = -0.5$. S5 Right → S6 (mines, terminal), $r = -10$: target -10; $Q(\mathrm{S5},\mathrm{R}) = -5.0$.

Episode 2. S4 Up $\to$ S1: $Q(\text{S4}, \text{U}) = -0.5$. S1 Right $\to$ S2: $Q(\text{S1}, \text{R}) = -0.5$. S2 Right $\to$ S3 (goal), $r = +10$: $Q(\text{S2}, \text{R}) = 5.0$.
Episode 3. S4 Up $\to$ S1: target $-1 + 0.9 \times 0 = -1$ (S1's best entry is still an untried 0); $Q(\text{S4}, \text{U}) = -0.5 + 0.5(-1 + 0.5) = -0.75$. S1 Right $\to$ S2: target $-1 + 0.9 \times 5.0 = 3.5$; $Q(\text{S1}, \text{R}) = -0.5 + 0.5(3.5 + 0.5) = \mathbf{1.5}$. S2 Right $\to$ S3: $Q(\text{S2}, \text{R}) = 5 + 0.5(10 - 5) = 7.5$.
Episode 4. S4 Right $\to$ S5: $Q(\text{S4}, \text{R}) = -0.75$. S5 Up $\to$ S2: target $-1 + 0.9 \times 7.5 = 5.75$; $Q(\text{S5}, \text{U}) = 2.875$. S2 Right $\to$ S3: $Q(\text{S2}, \text{R}) = 8.75$.

Q after 4 episodes	Up	Down	Left	Right
S1	0	0	0	1.50
S2	0	0	0	8.75
S4	−0.75	0	0	−0.75
S5	2.875	0	0	−5.00

What to notice. (1) The good news about the goal travels backwards *one step per episode*: S2 learned in episode 2, S1 in episode 3. (2) The minefield entry is already strongly negative after one visit, so a greedy agent in S5 will now go Up. (3) Untried actions still hold 0, which is *better* than the −0.75 of the tried moves from S4: a purely greedy agent would now waste time bumping into walls (Down and Left from S4 leave it in place). Exploration and many more episodes are needed before the table approaches Q^*.

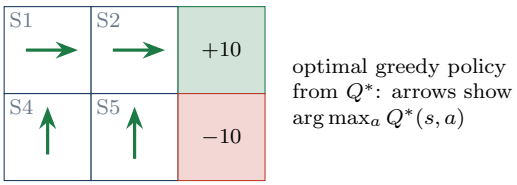


Figure B11: the policy that the Q-table converges to (from S4, Right is equally good).

24.8 Exploration versus exploitation

To learn, the agent must sometimes try actions that currently look worse (**explore**); to perform, it must use what it knows (**exploit**). The simplest rule is **ε-greedy**: with probability ε act at random, otherwise act greedily. With $\varepsilon = 0.1$ and four actions the greedy action is chosen with probability $0.9 + 0.1/4 = 0.925$. A common schedule starts at $\varepsilon = 1$ (pure exploration) and decays to a small floor such as 0.05; it is set to zero only when the trained policy is evaluated. Alternatives include optimistic initial values (start every Q high, so untried actions look attractive), softmax (Boltzmann) exploration, and upper-confidence bonuses that favour rarely tried actions.

24.9 On-policy and off-policy: SARSA versus Q-learning

Q-learning's target uses $\max_{a'} Q(s', a')$ — the value of the *best* next action — regardless of what the agent will actually do next. It is **off-policy**: it learns about the greedy policy while following an exploratory one. **SARSA** uses the action actually taken next, $r + \gamma Q(s', a')$, and so learns the value of the exploratory policy itself (**on-policy**). The difference matters near hazards: on a route that skirts a minefield, Q-learning learns the shortest path along the edge (optimal if the agent never slips), whereas SARSA learns that occasional random moves near the edge are costly and prefers a safer detour. For a system that will keep exploring in a dangerous environment, the on-policy answer is the more realistic one.

24.10 When does Q-learning work?

Tabular Q-learning converges to Q^* with probability one if every state–action pair is visited infinitely often and the learning rate decays appropriately ($\sum_t \alpha_t = \infty$, $\sum_t \alpha_t^2 < \infty$), provided the environment is a stationary MDP. Each condition is violated somewhere in defence practice: some states are rare (a surprise attack), the environment is partially observable (fog of war), and the opponent adapts (non-stationarity, Lecture 12).

24.11 From tables to networks: the deep Q-network

24.11.1 Why tables break

A table needs one row per state. The patrol grid has 4. Eight units on a 20×20 grid have $400^8 \approx 6.6 \times 10^{20}$ joint positions before health, ammunition or terrain are counted; a single 84×84 grey-scale screen has $256^{7,056}$ possible images. No table can be stored, and even if it could, almost every row would never be visited, so nothing would be learned about it. The agent must **generalise**: recognise that unseen states resemble seen ones.

24.11.2 Function approximation

Replace the table by a parameterised function $Q(s, a; \theta)$ — a neural network that takes the state (as a vector, map or image) and outputs one Q-value per action. States that look alike get similar values automatically. Learning now adjusts θ instead of individual table cells, by gradient descent on the squared TD error:

$$\mathcal{L}(\theta) = \mathbb{E}\Big[\big(y - Q(s, a; \theta)\big)^2\Big], \qquad y = r + \gamma \max_{a'} Q(s', a'; \theta^-).$$

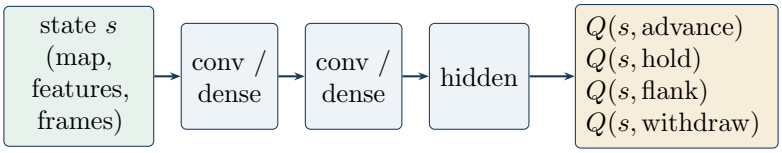


Figure B12: a deep Q-network outputs the value of every action in one forward pass; the agent picks the largest.

Worked example — the DQN loss on a mini-batch of two transitions

Transition 1: the agent chose *fire*, received $r = +2$, and the episode continues. The online network gives $Q(s, \text{fire}; \theta) = 5.0$; the target network gives next-state values $(4.0, 6.0, 3.0)$, so $y = 2 + 0.9 \times 6.0 = 7.4$ and the squared error is $(7.4 - 5.0)^2 = 5.76$.
Transition 2: the agent advanced into an ambush, $r = -10$, terminal. $Q(s, \text{advance}; \theta) = -4.0$; $y = -10$; squared error 36.0.
Mini-batch loss $= (5.76 + 36.0)/2 = 20.88$. Back-propagation pushes the first value up toward 7.4 and the second down toward -10, and — because the network shares parameters — adjusts the values of similar states too.

24.11.3 The two tricks that made DQN work

Naively combining Q-learning with a neural network is unstable: consecutive samples are highly correlated, and the target y moves every time θ changes, so the network chases its own tail. DQN (Mnih et al., 2015) added two stabilisers:

- **Experience replay.** Store transitions $(s, a, r, s', \text{done})$ in a large buffer (e.g. one million) and train on random mini-batches drawn from it. Random sampling breaks the temporal correlation, and each transition is reused many times.
- **Target network.** Compute y with a separate copy of the network, θ^-, that is frozen and only updated to equal θ every C steps (e.g. 10,000). The target stays still long enough to be learned.

Algorithm — deep Q-network

1. Initialise online network θ, target network $\theta^- \leftarrow \theta$, empty replay buffer $\mathcal{D}$.
2. For each step: choose a by ε-greedy on $Q(s, \cdot; \theta)$; execute; store $(s, a, r, s', \text{done})$ in $\mathcal{D}$.
3. Sample a mini-batch from $\mathcal{D}$; compute $y = r$ if done, else $r + \gamma \max_{a'} Q(s', a'; \theta^-)$.
4. Take a gradient step on $(y - Q(s, a; \theta))^2$.
5. Every C steps set $\theta^- \leftarrow \theta$.

With these changes one network architecture learned 49 Atari video games from raw pixels and the score alone, exceeding 75% of a professional human tester's score on 29 of them. The same recipe underlies the wargame agent of Lecture 10, and its network is trained by the backpropagation of Lecture 6.

24.11.4 Over-estimation and Double DQN

Taking a maximum over noisy estimates is biased upward. If four actions all truly have value 0 but the network's estimates carry independent unit noise, the expected maximum estimate is about $+1.03$, not 0. The agent becomes over-optimistic, especially about rarely tried actions. **Double DQN** uses the online network to *select* the next action and the target network to *evaluate* it:

$$y = r + \gamma\, Q\big(s', \arg\max_{a'} Q(s', a'; \theta);\ \theta^-\big),$$

which largely removes the bias. **Dueling** networks split the output into a state value $V(s)$ and action advantages $A(s, a)$, which helps when many actions have similar value. Lecture 10 develops both, together with prioritised replay and multi-step returns.

24.12 Beyond value-based RL

Value-based methods need a maximum over actions, which is awkward when actions are continuous (thrust, heading, bank angle). **Policy-gradient** methods instead adjust a parameterised policy $\pi_\theta(a \mid s)$ directly in the direction that makes high-return actions more likely; **actor–critic** methods combine a policy (actor) with a learned value function (critic); PPO and SAC are the workhorses (Lecture 11). **Model-based** methods learn or are given a model of the environment and plan with it — AlphaZero combined a learned value-and-policy network with Monte-Carlo tree search to master chess and Go. **Multi-agent** RL trains teams of agents together (Lecture 12).

24.13 Case study: a wargame agent

In the lecture's wargame (20×20 hex grid, 7 actions per unit), a DQN agent's win rate against a random opponent rose from 67% (10K episodes) to 85% (50K) and 91% (100K). (Lecture 10 reports a slower curve for its version of the agent, 38%, 76% and 91%; single training runs vary, and published learning curves should show several seeds.) A win rate against a random opponent says little about performance against a competent, adaptive one.

Pitfall — the reward is the specification

An RL agent optimises exactly the reward it is given, not the intention behind it. Rewarding enemy losses alone can teach reckless attacks; penalising own losses alone can teach the agent never to engage. Reward design is a command decision that must be reviewed like any other order (Lectures 10 and 12).

Check your understanding — reinforcement learning

(1) Using the optimal Q-table, what is $V^*(\text{S4})$? (2) In episode 3 why did $Q(\text{S4}, \text{Up})$ *fall* even though Up is an optimal action? (3) Why can DQN not simply use the online network to compute its own target?

Answers. (1) $V^*(s) = \max_a Q^*(s,a) = 6.2$, matching the return of the best route computed earlier. (2) Its target used S1's current best estimate, which was still 0 (the good news had not yet reached S1), so the target -1 was below the old value -0.5; with more episodes the value rises toward 6.2. (3) The target would shift with every update, making learning unstable; the frozen target network keeps the regression target fixed for C steps.

25 Deep Learning

This section is a first tour. Lectures 6–9 develop it in full: Lecture 6 explains why networks need hidden layers and works backpropagation by hand; Lecture 7 builds convolutional networks for imagery; Lectures 8 and 9 build the transformer, first by intuition and then with the full mathematics.

25.1 From the neuron to the network

An artificial neuron computes a weighted sum of its inputs plus a bias and passes it through a non-linear **activation function**:

$$z = \sum_i w_i x_i + b, \qquad a = \sigma(z).$$

Rosenblatt's **perceptron** (1958) used a step function and could learn any linearly separable classification; Minsky and Papert (1969) showed that a single layer cannot represent XOR, which contributed to the first AI winter. Stacking layers removes the limitation: a **multi-layer perceptron** (MLP) composes $h^{(1)} = \sigma(W^{(1)}x + b^{(1)})$, $h^{(2)} = \sigma(W^{(2)}h^{(1)} + b^{(2)})$, …, and the output layer produces a prediction. Without non-linear activations, any stack of layers collapses to a single linear map; the non-linearity is what gives depth its power.

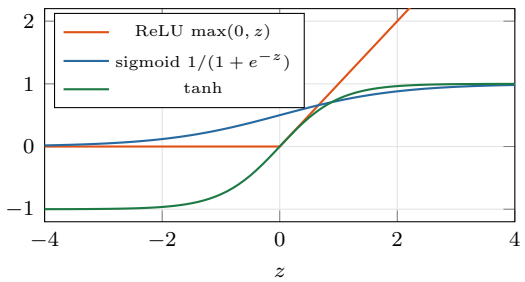


Figure B13: common activations. ReLU is the default in hidden layers; sigmoid and softmax produce probabilities at the output.

Definition — universal approximation (Hornik, 1989)

A feed-forward network with at least one hidden layer and enough units can approximate any continuous function on a compact set to arbitrary accuracy. The theorem guarantees that a good network exists, not that training will find it or that it will generalise.

25.2 Training: forward pass, loss, backward pass

Training repeats three steps on mini-batches: a **forward pass** computes predictions; a **loss** compares them with the targets; a **backward pass** (back-propagation) computes the gradient of the loss with respect to every weight by applying the chain rule from the output back to the input; then gradient descent updates the weights. Back-propagation is efficient because each layer reuses the gradient already computed for the layer above it.

Worked example — a complete forward and backward pass by hand

Network: 2 inputs → 2 sigmoid hidden units → 1 sigmoid output; binary cross-entropy loss. Input $x = (1.0,\ 0.5)$, target $y = 1$.

Weights: $W^{(1)} = \begin{pmatrix} 0.4 & -0.2 \\ 0.3 & 0.6 \end{pmatrix}$, $b^{(1)} = (0, 0)$; $w^{(2)} = (0.7,\ -0.5)$, $b^{(2)} = 0.1$.

Forward. $z^{(1)} = W^{(1)}x = (0.4 - 0.1,\ 0.3 + 0.3) = (0.3,\ 0.6)$; $h = \sigma(z^{(1)}) = (0.574,\ 0.646)$.
$z^{(2)} = 0.7(0.574) - 0.5(0.646) + 0.1 = 0.179$; $\hat{y} = \sigma(0.179) = 0.545$. Loss $= -\ln 0.545 = 0.608$.

Backward. For sigmoid output with cross-entropy, $\delta^{(2)} = \hat{y} - y = -0.455$.
Output-layer gradients: $\partial\mathcal{L}/\partial w^{(2)} = \delta^{(2)}h = (-0.262,\ -0.294)$; $\partial\mathcal{L}/\partial b^{(2)} = -0.455$.
Hidden deltas: $\delta_j^{(1)} = \delta^{(2)} w_j^{(2)}\, h_j(1 - h_j)$, giving $\delta^{(1)} = (-0.078,\ +0.052)$.
Hidden-layer gradients: $\partial\mathcal{L}/\partial W_{jk}^{(1)} = \delta_j^{(1)} x_k$, i.e. $\begin{pmatrix} -0.078 & -0.039 \\ 0.052 & 0.026 \end{pmatrix}$.

Update with $\alpha = 1$: $w^{(2)} = (0.962,\ -0.206)$, $b^{(2)} = 0.555$, $W^{(1)} = \begin{pmatrix} 0.478 & -0.161 \\ 0.248 & 0.574 \end{pmatrix}$. The new prediction is $\hat{y} = 0.731$ and the loss falls to 0.313. Notice the signs: the hidden unit connected by a negative weight has its incoming weights pushed *down*, because increasing its activation would have lowered the output.

25.3 Making training work

- **Optimisers.** Plain SGD; *momentum*, which accumulates a velocity $v \leftarrow \beta v + \nabla\mathcal{L}$, $\theta \leftarrow \theta - \alpha v$ and rolls through small bumps; *Adam*, which scales each parameter's step by running estimates of the gradient's mean and variance, so its first step has size about α regardless of the gradient's scale. Adam with $\alpha \approx 10^{-3}$ is the usual starting point.
- **Initialisation.** Weights start small and random, scaled to the layer width (Xavier/Glorot for tanh, He for ReLU), so that signals neither vanish nor explode as they pass through many layers.
- **Vanishing and exploding gradients.** The chain rule multiplies many factors; sigmoid derivatives are at most 0.25, so in deep sigmoid networks gradients shrink exponentially toward the input. ReLU activations, careful initialisation, normalisation layers and *residual connections* ($h_{l+1} = h_l + F(h_l)$, which give gradients a direct path) made networks of 100+ layers trainable.
- **Regularisation.** Weight decay (L2), *dropout* (randomly silence a fraction of units during training so that no unit can rely on another), *early stopping* (halt when validation loss rises), and *data augmentation* (rotations, crops, noise, simulated clutter) all reduce overfitting.

- **Batch and layer normalisation** rescale activations to zero mean and unit variance inside the network, which stabilises and speeds up training.

25.4 Convolutional neural networks

Images and SAR chips have local structure: nearby pixels are related, and a vehicle is a vehicle wherever it appears. A **convolutional layer** slides a small kernel (e.g. 3×3) over the image and computes a weighted sum at each position, producing a *feature map*. The same kernel weights are shared across all positions, which gives translation equivariance and drastically reduces the parameter count.

Worked example — a vertical-edge detector

A 5×5 image is dark (0) in its left three columns and bright (9) in its right two. Kernel $K = \begin{pmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{pmatrix}$. At the top-left position the window is all zeros, output 0. One step right the window covers columns 2–4: $(0+0+0)\times 1 + (0)\times 0 + (9+9+9)\times(-1) = -27$. The full 3×3 output is

$$\begin{pmatrix} 0 & -27 & -27 \\ 0 & -27 & -27 \\ 0 & -27 & -27 \end{pmatrix}:$$

large magnitude exactly where the brightness changes. Early CNN layers learn kernels like this one automatically.

Parameter economy. A convolutional layer with 32 kernels of size 3×3 on a 3-channel image has $3 \cdot 3 \cdot 3 \cdot 32 + 32 = 896$ parameters regardless of image size. A dense layer connecting a $32 \times 32 \times 3$ image to just 32 units needs $3{,}072 \cdot 32 + 32 = 98{,}336$. **Pooling** (taking the maximum or mean over small windows) shrinks feature maps and adds tolerance to small shifts. Stacking convolution and pooling layers grows the **receptive field**, so deeper units respond to larger patterns: edges $\rightarrow$ corners and shadows $\rightarrow$ vehicle parts $\rightarrow$ vehicles $\rightarrow$ formations.

25.5 Recurrent networks and LSTMs

Radar tracks, log streams and sensor time series are sequences. A **recurrent network** carries a hidden state forward in time, $h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$, so its output can depend on the whole history. Plain RNNs forget quickly because of vanishing gradients through time. The **LSTM** adds a memory cell c_t and three gates — forget f_t, input i_t and output o_t, each a sigmoid of $[h_{t-1}, x_t]$ — with $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$ and $h_t = o_t \odot \tanh(c_t)$. Because the cell is updated additively, information and gradients can persist over hundreds of steps. LSTM autoencoders detect insider threats from activity sequences in Lecture 5.

25.6 Attention and Transformers

Transformers replaced recurrence with **self-attention**: every element of a sequence computes how much to attend to every other element and takes a weighted average of their information. Each token i is projected into a query q_i, a key k_i and a value v_i; then

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

Worked example — attention among three tokens

Three report fragments have 2-D embeddings $x_1 = (1,0)$, $x_2 = (0,1)$, $x_3 = (1,1)$; for simplicity queries and keys equal the embeddings, and values are $v_1 = (1,0)$, $v_2 = (0,2)$, $v_3 = (1,1)$. Scaled scores for token 1 are $(1,0,1)/\sqrt{2} = (0.707, 0, 0.707)$; the softmax gives weights (0.401, 0.198, 0.401), so token 1's output is $0.401v_1 + 0.198v_2 + 0.401v_3 = (0.802, 0.797)$. Token 3, similar to both, spreads its attention $(0.248, 0.248, 0.504)$. Each output is a context-aware mixture of the whole sequence, computed in parallel for all tokens.

Multi-head attention runs several such computations side by side, positional encodings tell the model the order of tokens, and dozens of layers are stacked. Transformers now dominate text, are strong on images (vision transformers) and are used to fuse heterogeneous sensor streams.

25.7 Autoencoders and generative networks

An **autoencoder** compresses its input to a small code and reconstructs it; trained on normal data, it reconstructs anomalies poorly (Lecture 5). A **variational autoencoder** makes the code probabilistic so that new samples can be generated. A **generative adversarial network** trains a generator to produce samples that a discriminator cannot tell from real ones, $\min_G \max_D \; \mathbb{E}[\log D(x)] + \mathbb{E}[\log(1 - D(G(z)))]$; it is used for synthetic training imagery. **Diffusion models** learn to reverse a gradual noising process and now produce the highest-quality synthetic images.

25.8 Why depth helps — and what it costs

Deep networks learn hierarchical features. For SAR imagery: layer 1 edges and shadow boundaries; layer 2 corner reflectors and shapes; layer 3 vehicle silhouettes; layer 4 vehicle type (tank vs APC vs truck); layer 5 formations; deeper layers context. Architectures match data types: **CNNs** for images and SAR; **RNNs/LSTMs** for sequences such as radar tracks; **Transformers** for text and multi-source fusion; **GANs** and diffusion models for synthetic data. Empirical **scaling laws** (Kaplan et al., 2020) show that performance improves predictably with data, parameters and compute.

The costs are data hunger, compute, opacity and fragility. **Explainability** tools — saliency maps and Grad-CAM, which highlight the image regions that drove a decision, and SHAP values, which attribute a prediction to input features — help detect shortcut learning (the "snow means wolf" error of Part A). They explain the model, not the world.

25.9 Adversarial examples

Deep networks can be fooled by small, deliberately crafted perturbations. The fast gradient sign method perturbs an input in the direction that most increases the loss:

$$x_{\text{adv}} = x + \varepsilon \,\text{sign}\big(\nabla_x \mathcal{L}(x, y)\big).$$

With ε too small to be visible, a correctly classified image can be confidently misclassified. Physical versions exist: printed patches on vehicles and patterns on clothing that defeat detectors. In defence the adversary is real and motivated, so robustness testing, adversarial training and multi-sensor confirmation are requirements, not refinements.

Check your understanding — deep learning

(1) Why does a network with linear activations gain nothing from extra layers? (2) In the hand-worked example, why did $b^{(2)}$ increase? (3) A detector trained on daytime electro-optical

imagery fails at night: which earlier concept explains this, and what would you do?
Answers. (1) A composition of linear maps is linear, so the stack equals one layer. (2) Its gradient $\hat{y} - y = -0.455$ was negative (the output was too low), so gradient descent increased it. (3) Distribution shift (Section 28.3); add night and infrared data or augmentation, test on the deployment condition, and fuse with a sensor unaffected by light.

26 Generative AI: LLMs and RAG

26.1 Tokens, embeddings and next-token prediction

A large language model (LLM) reads and writes **tokens** — word pieces from a fixed vocabulary of typically 30,000–200,000 entries ("reconnaissance" might be split into "recon", "naiss", "ance"). Each token is mapped to an **embedding**, a vector of a few thousand numbers learned so that tokens used in similar contexts have similar vectors. A stack of Transformer decoder layers (Section 25) turns the sequence of embeddings into a probability distribution over the next token:

$$P(w_t \mid w_1, \ldots, w_{t-1}; \theta).$$

Training maximises the probability of the actual next token across trillions of tokens of text — self-supervised learning at enormous scale. Generation repeats the prediction: sample a token, append it, predict again. GPT-3 has 175 billion parameters trained on about 300 billion tokens; at scale, models show in-context learning (following examples given in the prompt), step-by-step reasoning and code generation.

26.2 Sampling and temperature

The model's final layer produces scores (logits) z_i for every token, and the probabilities are

$$p_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)},$$

where T is the **temperature**.

Worked example — the effect of temperature

Three candidate next words have logits 2.0, 1.0 and 0.1. At $T = 1$ their probabilities are 0.659, 0.242, 0.099. At $T = 0.5$ they sharpen to 0.864, 0.117, 0.019; at $T = 2$ they flatten to 0.502, 0.304, 0.194. Low temperature gives consistent, conservative text (preferred for reports and extraction); high temperature gives variety (brainstorming red-team ideas) and more errors.

26.3 From a text predictor to an assistant

1. **Pre-training** on a broad corpus produces a model that continues text but does not reliably follow instructions.
2. **Supervised fine-tuning** on examples of instructions and good responses teaches the assistant format.
3. **Preference optimisation** — reinforcement learning from human feedback (RLHF) or direct preference optimisation — trains the model toward responses that human raters prefer: helpful, honest, harmless. A reward model learned from rankings plays the role of the reward in Section 24.

4. **Adaptation** to a domain uses full fine-tuning, parameter-efficient methods such as LoRA (small low-rank adapter matrices trained while the base weights stay frozen), or simply better prompts and retrieved context.

The **context window** — the number of tokens the model can attend to at once, from thousands to over a million — limits how much material can be placed in a single prompt.

26.4 Why LLMs hallucinate

An LLM is trained to produce *plausible* continuations, not *verified* ones. When the training data contained the answer many times, plausible and true coincide. When the question concerns rare, recent, classified or local facts, the most plausible continuation may be fluent fiction: an invented report number, a non-existent regulation, a confident but wrong coordinate. The model has no built-in signal that distinguishes recalling from inventing, and its tone is equally confident in both cases. Hallucination is therefore a structural property, not an occasional bug, and the remedies are structural: ground the answer in retrieved sources, require citations, and have a human verify claims that matter.

26.5 Prompting

How a request is phrased changes the output. Useful practices: state the role, task, audience and format explicitly; supply the source material rather than relying on the model's memory; give one or two examples of the desired output (few-shot prompting); ask for step-by-step reasoning on analytical tasks; ask the model to say "not in the sources" when the answer is absent; and request quotations or citations that a reader can check.

26.6 Retrieval-augmented generation in detail

RAG answers a query from a curated document store instead of from the model's memory.

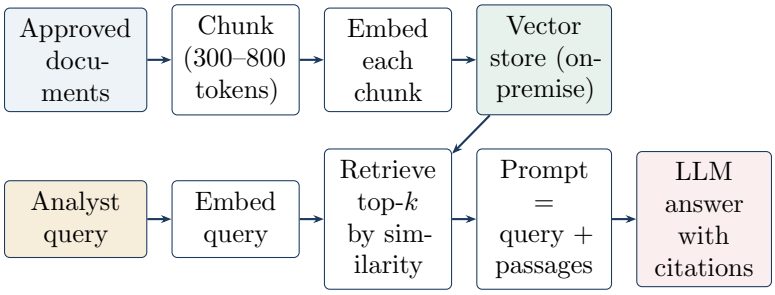


Figure B14: indexing (top) happens once; retrieval and generation (bottom) happen for every query.

Indexing. Documents are split into overlapping chunks, each converted to an embedding vector by an embedding model, and stored with its source reference and security marking.
Retrieval. The query is embedded with the same model; the store returns the k chunks with the highest **cosine similarity** $\cos(q, d) = q^\top d / (\|q\| \|d\|)$, often combined with keyword search and re-ranked by a second model.

Worked example — retrieving by cosine similarity

A query embedding $q = (0.8, 0.1, 0.5)$ is compared with three chunk embeddings: $d_1 = (0.7, 0.2, 0.6)$ gives 0.983; $d_2 = (0.1, 0.9, 0.2)$ gives 0.307; $d_3 = (0.6, 0.0, 0.3)$ gives 0.990. With $k = 2$ the system retrieves d_3 and d_1. Note that d_3 wins although its components are smaller: cosine similarity compares direction (topic), not length.

Generation. The retrieved passages and the query are assembled into a prompt instructing the model to answer only from the passages and to cite them. In the lecture's comparison, factual accuracy rose from 73.2% to 94.6%, hallucination fell from 18.4% to 3.2%, and 91.8% of answers became citable, at the cost of about 1.3 s extra latency.

Evaluating a RAG system needs separate measures for retrieval (did the right passages come back? — recall@k) and generation (is every claim supported by the retrieved passages? — *faithfulness*; does the answer address the question? — *relevance*). A system can retrieve well and still misread, or answer fluently from the wrong passage.

26.7 Security of generative systems

- **Data leakage.** Queries and documents sent to an external service leave the security boundary. A defence RAG runs entirely on-premise — local embedding model, vector store and LLM — with encryption and access control.
- **Access control in retrieval.** The retriever must enforce the user's clearance and need-to-know; otherwise the LLM becomes a way to read documents the user could not open directly.
- **Prompt injection.** Text inside a retrieved document ("ignore previous instructions and ...") can hijack the model. Treat retrieved text as data, restrict the actions the model can trigger, and keep a human in the loop for consequential actions.
- **Poisoning.** Planting false documents in the corpus, or in pre-training data, corrupts answers; provenance tracking and curation are the defence.

26.8 Defence uses and risks

Uses: summarising hundreds of intelligence reports into a brief, narrating sensor outputs as threat assessments, drafting courses of action, translating intercepts, explaining malware, generating red-team scenarios for wargames. Risks: **hallucination**; data poisoning; lack of interpretability; latency of seconds (unsuitable for real-time engagement); leakage of classified data to cloud services; and **automation bias**, the tendency of busy users to accept fluent output without checking. LLM **agents** that call tools (search, databases, planning software) multiply both the usefulness and the risk: every tool an agent can use is an action an injected instruction can trigger.

Key point

Use generative AI to draft, summarise and search; keep humans responsible for verifying facts and for every decision with operational consequences.

27 Defence Applications and Wargaming

AI applications span six domains: ISR (image and signal analysis, multi-source fusion), cyber operations (intrusion detection, malware classification), autonomous systems (UAV path planning, swarms, mine detection), logistics (demand forecasting, predictive maintenance), wargaming and simulation (OPFOR agents, terrain analysis, attrition modelling, COA generation) and command and control (force tracking, commander briefings). Data flows ISR $\rightarrow$ C2 $\rightarrow$ wargaming $\rightarrow$ autonomous systems along the sensor-to-shooter chain.

In wargaming specifically: RL-based **computer-generated forces** reached about 78% of an expert human OPFOR's effectiveness at roughly a thousandth of the staffing cost. **Lanchester's laws** $dR/dt = -\alpha B$, $dB/dt = -\beta R$ model attrition; a random forest adding terrain, weather,

morale and EW raised explained variance from $R^2 = 0.61$ to 0.87 over 1,200 engagements. CNN **terrain analysis** reached 94.8% accuracy at 11 ms per patch — 12 minutes instead of 4 hours per 1,000 km^2. AI **course-of-action generation** produced three options in 45 s, though rated acceptable less often than staff COAs (72% vs 89%). AI in wargaming matures from scripted AI to adaptive RL, multi-agent coalitions, LLM-reasoning advisers and, speculatively, autonomous campaign planning.

Pitfall — the simulation-to-reality gap

A 91% win rate in a simulator is not 91% in the field. Fog of war, friction, sensor noise and an adapting adversary must be modelled, and simulation results must be presented with explicit caveats.

28 Data Pitfalls in AI Pipelines

28.1 Survivorship bias and selection

Training only on battles won predicted a 74% win rate where the actual rate was 53%. Seek defeat records, after-action reports and failure analyses, and weight rare failures appropriately.

28.2 Overfitting and the bias–variance trade-off

Derivation — bias–variance decomposition

For a target $y = f(x) + \varepsilon$ with noise variance σ^2 and a learned predictor $\hat{f}$, the expected squared error at x is

$$\mathbb{E}\big[(y - \hat{f}(x))^2\big] = \underbrace{\big(\mathbb{E}[\hat{f}(x)] - f(x)\big)^2}_{\text{bias}^2} + \underbrace{\mathbb{E}\big[(\hat{f}(x) - \mathbb{E}\hat{f}(x))^2\big]}_{\text{variance}} + \underbrace{\sigma^2}_{\text{irreducible}} .$$

Simple models have high bias; complex models have high variance. Validation error is minimised in between.

Worked example — mine detection

On 800 Afghan samples, a simple CNN (200K parameters) scored 88.4% train, 86.2% Afghan test and 82.7% Iraq test. A deep ResNet (25M parameters) scored 97.3%, 94.1% and only **58.7%** in Iraq. The larger model memorised Afghan soil signatures; the smaller model generalised.

28.3 Distribution shift

Deployment data differ from training data: day vs night, desert vs jungle, 2021 vs 2026 threat profiles. The two-sample Kolmogorov–Smirnov statistic $D = \max_x |F_{\text{new}}(x) - F_{\text{train}}(x)|$ measures the largest gap between the empirical distribution functions of a feature; raise an alert when D exceeds a threshold. Remedies: regularisation, augmentation, domain-adaptive training and — above all — testing on data from the deployment domain.

28.4 Training-data bias and label noise

With 95% non-threat and 5% threat examples, a model that always says "non-threat" scores 95% accuracy and detects nothing; fixes include class weighting and SMOTE oversampling. Selection bias in collection (e.g. face-recognition error rates of 34% for darker-skinned faces when trained on unrepresentative data, MIT 2018) and analyst confirmation bias in labels corrupt the ground truth. Label noise degrades accuracy steadily: 0% noise 97.2%; 5% → 93.8%; 10% → 88.1%; 20% → 76.4%; 30% → 61.7%. Adversarial **data poisoning** of just 3% of training data flipped 94% of predictions on a target class; training pipelines must be signed and monitored.

29 Summary and Glossary

29.1 Summary

- Data analysis fails through Simpson's paradox, misread significance tests, misleading correlations, regression to the mean, ignored base rates and missing data — each with real and costly defence precedents.
- AI is the broad field; ML learns from data; DL uses deep networks. Today's AI is narrow.
- Classical AI (rules, search) is transparent but brittle and does not scale; ML learns from data.
- Five paradigms: supervised, unsupervised, semi-supervised, self-supervised, reinforcement.
- Supervised learning minimises a loss on labelled data by gradient descent and must be judged on held-out data with metrics that reflect operational costs.
- Unsupervised learning finds clusters, densities and low-dimensional structure; its results are hypotheses to be validated.
- Reinforcement learning learns values $Q(s, a)$ from rewards through the Bellman equation; a Q-table works for small problems, and a deep Q-network with replay and a target network generalises to large ones.
- Deep networks learn hierarchical features by back-propagation; CNNs, LSTMs and Transformers match images, sequences and text; all are vulnerable to adversarial inputs.
- Generative AI and RAG are powerful for intelligence synthesis but hallucinate unless grounded.
- Every AI result must be validated on the deployment domain and interpreted by an accountable human.

29.2 Glossary

Activation function	Non-linear function applied to a neuron's weighted input (ReLU, sigmoid, tanh).
Agent	System that perceives an environment and acts on it; specified by PEAS.
Alpha–beta pruning	Skipping game-tree branches that cannot affect the minimax decision.
Automation bias	Over-reliance on automated recommendations.
Back-propagation	Chain-rule computation of loss gradients for every network weight.
Base rate	Prior frequency of a condition in the population of interest.

Bayesian network	Graph of variables with conditional probabilities; combines evidence by Bayes' rule.
Bellman equation	Recursive relation: value = immediate reward + discounted value of what follows.
Calibration	Agreement between stated confidence and observed accuracy.
Confounder	Variable affecting both group assignment and outcome.
Convolution	Sliding a shared kernel over an input to produce a feature map.
Cross-validation	Rotating validation folds to estimate performance and tune hyper-parameters.
Deep Q-network (DQN)	Neural network approximating $Q(s, a)$, trained with replay and a target network.
Distribution shift	Difference between training and deployment data distributions.
Effect size	Magnitude of a difference or relationship, independent of sample size.
ε-greedy	Act randomly with probability ε, otherwise take the best-valued action.
Embedding	Learned vector representing a token, image or object; similar items have similar vectors.
Experience replay	Buffer of past transitions sampled at random for training.
Gradient descent	Iterative update $\theta \leftarrow \theta - \alpha \nabla \mathcal{L}$.
Hallucination	Fluent but unsupported or false output from a generative model.
Loss function	Measure of the cost of a prediction error; training minimises its average.
Markov decision process	(S, A, P, R, γ): the formal model of sequential decision-making.
Null hypothesis H_0	Default "no effect" claim tested by a significance test.
Overfitting	Fitting noise in training data, hurting performance on new data.
Policy	Rule mapping states to actions (or action probabilities).
Power	Probability that a test detects a real effect $(1 - \beta)$.
PPV (precision)	Probability that a positive alarm is a true positive.
p-value	Probability of data at least as extreme as observed, if H_0 were true.
Q-table	Table of $Q(s, a)$ values, one row per state and one column per action.
Regression to the mean	Tendency of extreme measurements to be followed by less extreme ones.
Retrieval-augmented generation	Answering from retrieved, citable passages rather than from model memory.
Self-attention	Each sequence element forms a weighted mixture of all others by query–key similarity.
Shortcut learning	A model exploiting spurious cues correlated with labels.
Silhouette	$(b - a)/\max(a, b)$: how well a point sits in its cluster.
Simpson's paradox	Reversal of a comparison when subgroups are combined.
Survivorship bias	Distortion from analysing only cases that passed a selection filter.
Target network	Frozen copy of a DQN used to compute stable learning targets.
TD error	$r + \gamma \max_{a'} Q(s', a') - Q(s, a)$: the surprise that drives Q-learning.
Temperature	Parameter that sharpens $(T < 1)$ or flattens $(T > 1)$ a softmax distribution.
Type I / II error	False positive / false negative in a hypothesis test.